Speed is a shape

(why React is slow)



Sufian Rhazi

NY UX Speed and Web Performance Meetup

Sep 17, 2026

Hi! I'm Sufian (sue-fee-ann / souf-yann / سفيان)

Love a good problem

A bit shy & soft-spoken

Been writing software for ~20 years
...split across ~8 companies 😳

Currently working at Cloudflare

I still enjoy programming!

But it all blurs together: it's all data plumbing

Software doesn't solve problems, people do



I've done a lot of different things

Ops & backend systems
email anti-spam



frontend infra, design systems, eng manager
e-commerce buyer & seller



"full stack"
drugs & electronic health records



"full stack"
wysiwyg website builder





frontend, backend, 3d engine
dress-up and chat



frontend product
business chat

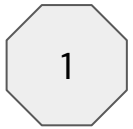


"full stack"
observability platform

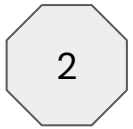


Mostly backend so far
internet infrastructure

Some things I've learned



To make it fast,
do less work



To make it small,
do less work



To make it reliable,
do less work

Thanks for coming!

(do less work)

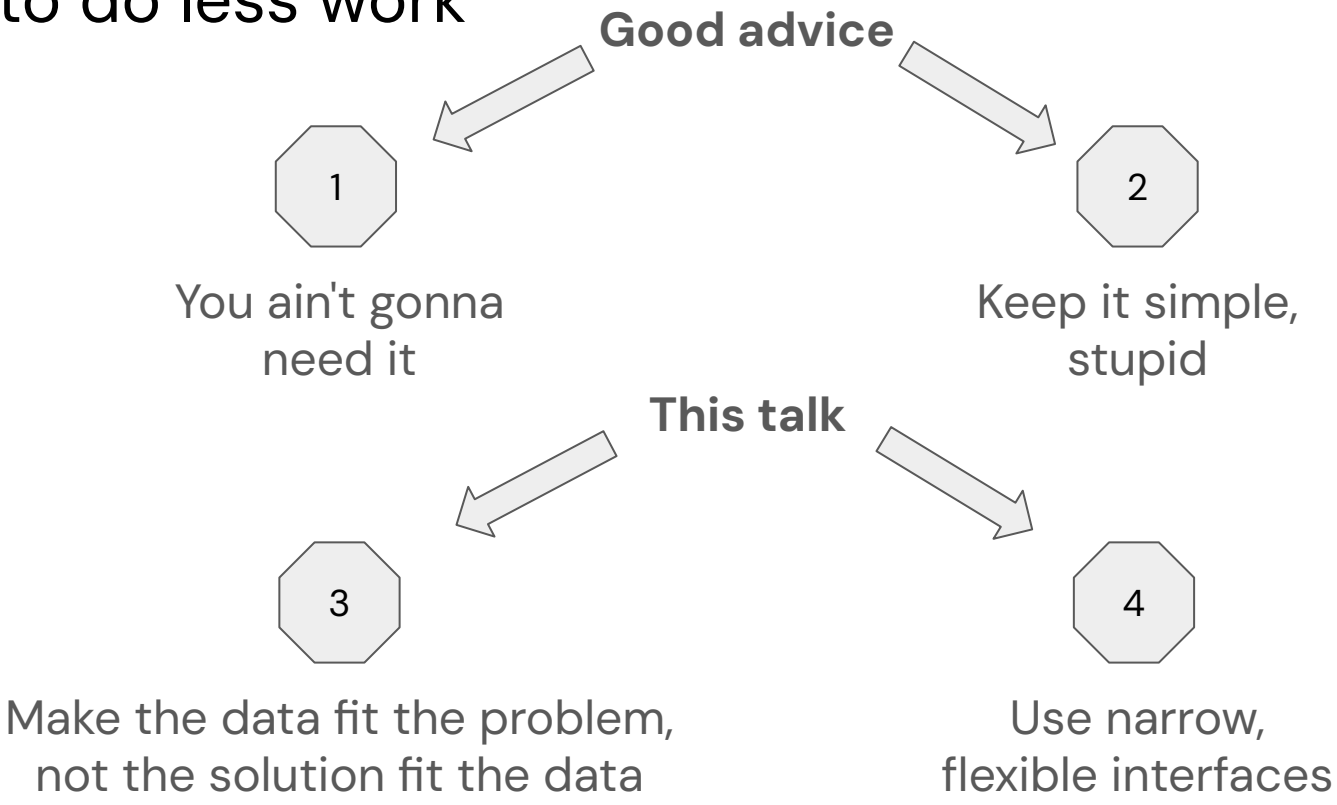


Sufian Rhazi

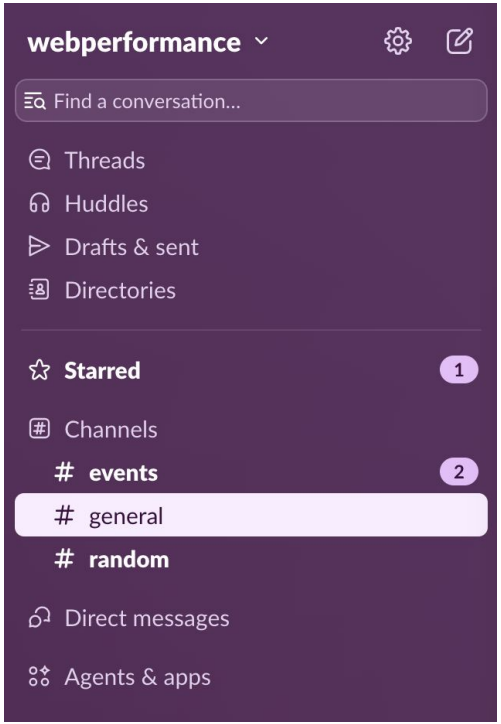
NY UX Speed and Web Performance Meetup

Sep 17, 2026

How to do less work



An example: The Channel Sidebar in Slack



This is the channel sidebar

Mostly static UI

Visually controlled by CSS (toggle classNames)

Information-dense, accessible

People somehow have thousands of channels

All connected to a redux store

An example: The Channel Sidebar in Slack



Redux store:

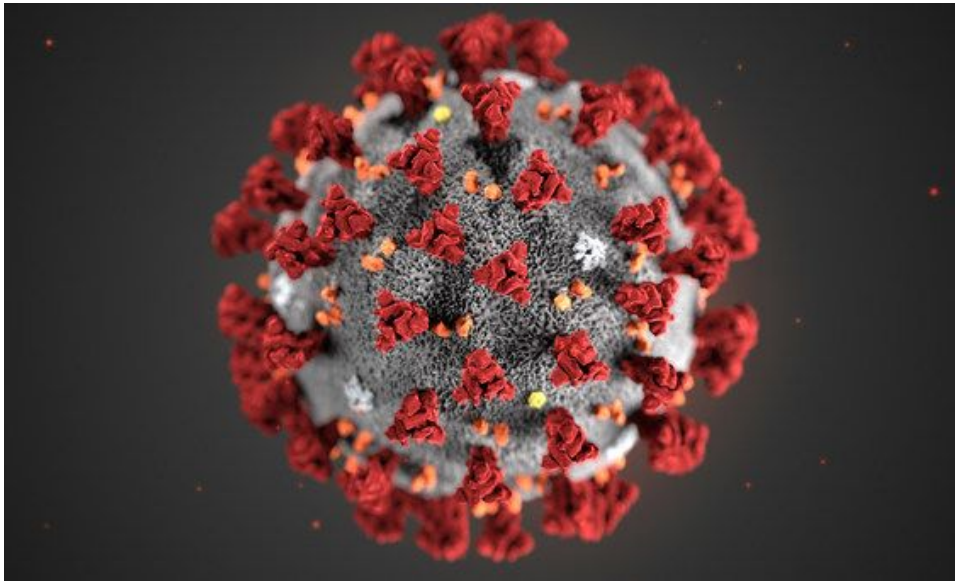
A big "channel id -> channel object" map

The app upserts channels when their attributes change (mention count, unread count, last message timestamp, etc...)

When a single channel gets a new message:

- Redux: "hey, I changed"
- React: Redux changed, I'll re-render 300 components
- User: Hmm, this app is laggy
- React: Ok, looks like nothing changed

Oh wait, this was in 2020



Uh-oh, a pandemic!

What should I do with all this anxiety and all this time on my hands?

I know, I'll make a frontend framework!

- Sufian, a totally normal, well-balanced person

So I made a hobby framework. It was fun, I like it, and I'm here to tell you about how changing the shape of a framework's API can impact an application's performance.

Agenda

1. What does React *do*?

2. Does it *need* to do that?

3. What does Gooey do instead?

Facebook? Boo!

hooks?!

JSX!

I have a confession...

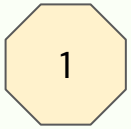
I **do not** like React

compilers!?

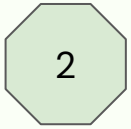
weird rules!

popular?!

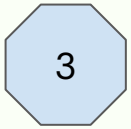
What does React do?



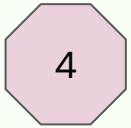
Transform JSX → VDOM



Render VDOM → HTML



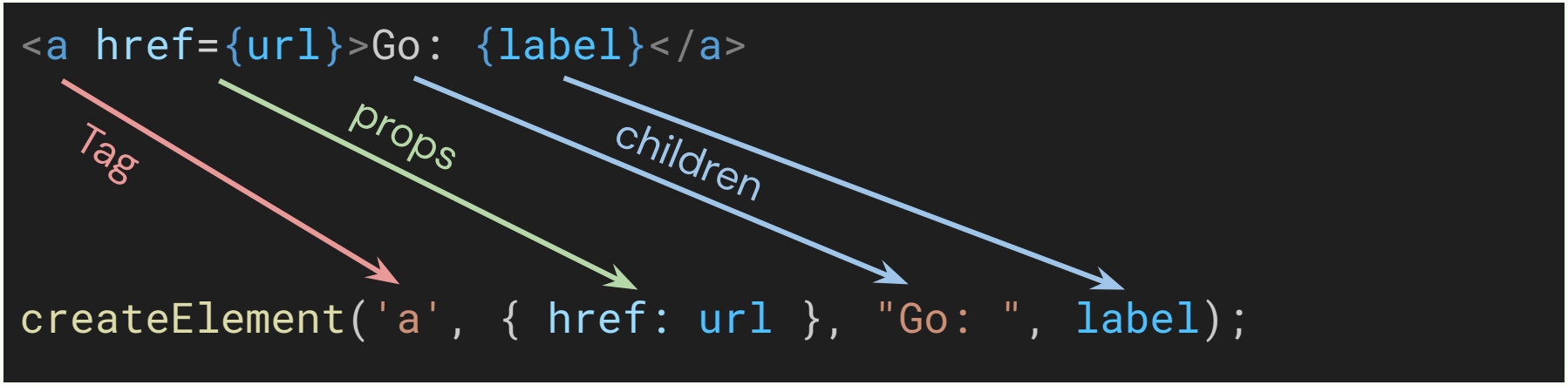
Rerender on state change



Diff $VDOM_a$ ↔ $VDOM_b$
& Apply to HTML

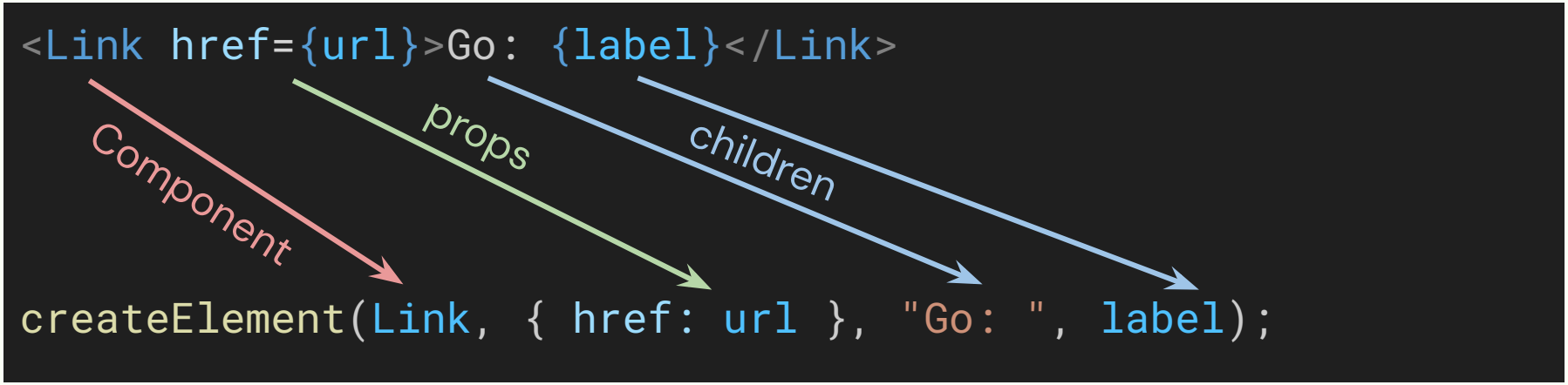
1 **Transform** JSX → VDOM

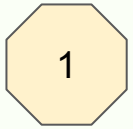
JSX is a fancy function call



1 **Transform** JSX → VDOM

Just a function call: nothing special about components/intrinsic elements





Transform JSX → VDOM

Just a function call

This step is *fast*

VDOM is "just" an object

it holds enough information & state
to RENDER & RERENDER

Pedantic note: technically there's a difference between what createElement returns and "fiber" nodes ("real" VDOM nodes), but I'm intentionally simplifying this to get a point across: JSX is just a function, and that function is pretty cheap.
Both creating a fiber and a "react element" are fast.

What does React do?

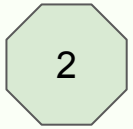
👍 FAST

1 **Transform** JSX → VDOM

2 **Render** VDOM → HTML

3 **Rerender** on state change

4 **Diff** $VDOM_a$ ↔ $VDOM_b$
& Apply to HTML

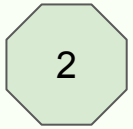


Render VDOM → HTML

Intrinsic elements render to plain DOM

```
createElement('a', { href: url }, "Go: ", label);
→ { tag: "a", props: { href: url }, children: ["Go: ", label] }
```

```
const element = document.createElement('a');
element.setAttribute('href', url);
element.replaceChildren('Go: ', label);
```



Render VDOM → HTML

Components are functions

Component functions use hooks, which are weird!

To render a component:

1. "Run" the Component with hook state
   (and save it for later)
2. Render the result, some more JSX
   (this may render additional components)

2 **Render** VDOM → HTML

```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```

How to "run" a component with hook state:

1. Make a list of slots & pointer to current slot
2. Each hook call has state in current slot
3. Update pointer after each hook call

2 **Render** VDOM → HTML

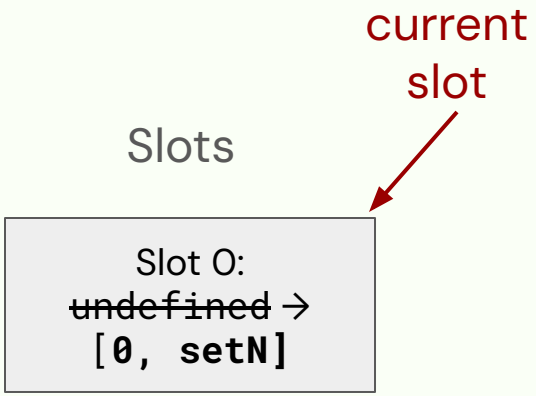
```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```

Slots

no current slot

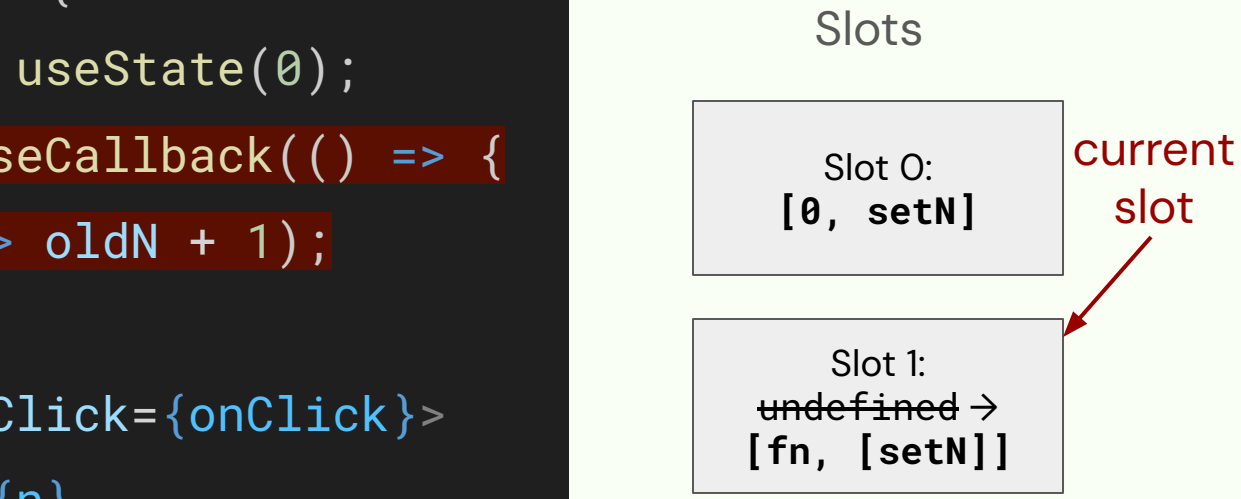
2 **Render** VDOM → HTML

```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```



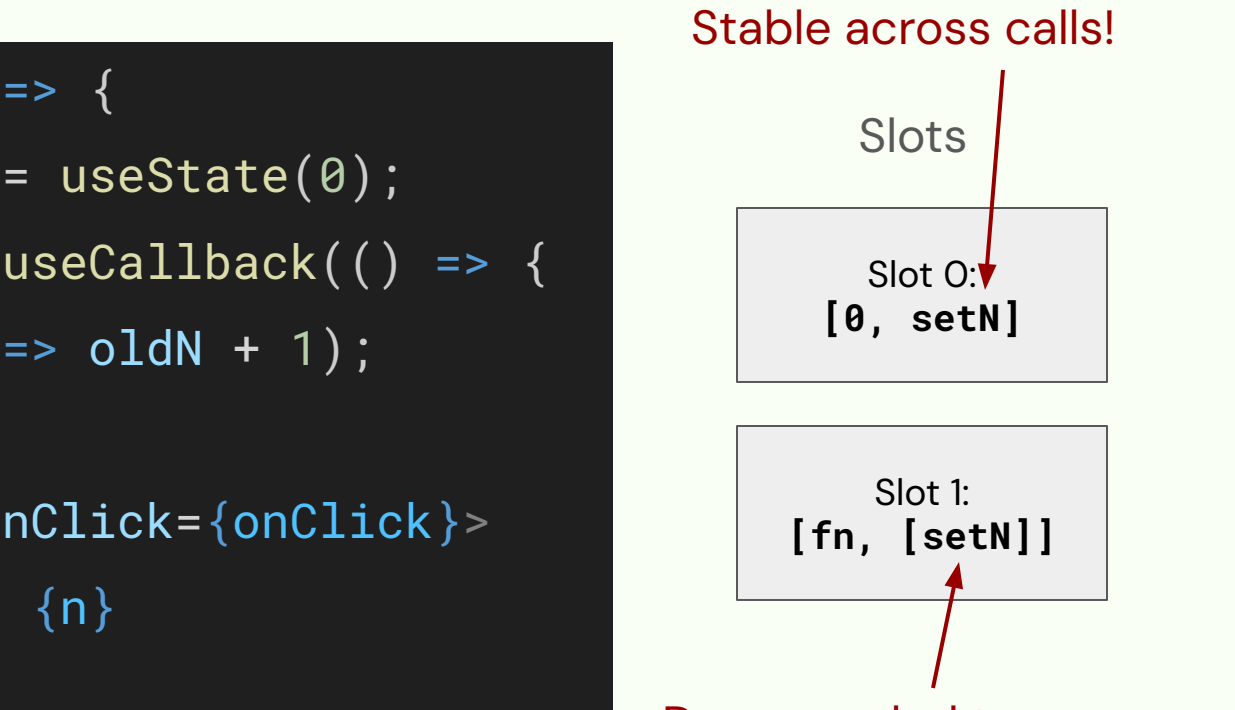
2 **Render** VDOM → HTML

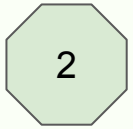
```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```



2 **Render** VDOM → HTML

```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```





Render VDOM → HTML

Intrinsic elements become DOM nodes

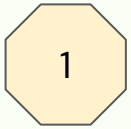
Components allocate hook state & "run" hooks

Hook state saved in VDOM for re-render & unmount

Overall, this step is also fast
(assuming your components are fast)

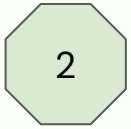
What does React do?

👍 FAST

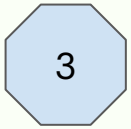


Transform JSX → VDOM

👍 FAST

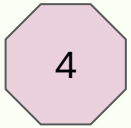


Render VDOM → HTML



Rerender VDOM on state change

🤔 Are *these* slow?



Diff $VDOM_a$ ↔ $VDOM_b$
& Apply to HTML

3 **Re-render** on state change

```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```

1. When this is called...

Slots

Slot 0:
`[0, setN]`

Slot 1:
`[fn, [setN]]`

3 **Re-render** on state change

```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```

2. Callback is called

1. When this is called...

Slots

Slot 0:
`[0, setN]`

Slot 1:
`[fn, [setN]]`

3 **Re-render** on state change

```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```

2. Callback is called

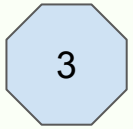
3. Which updates state!
Mark VDOM for re-render

1. When this is called...

Slots

Slot 0:
[~~0~~ → **1**, setN]

Slot 1:
[fn, [setN]]



Re-render on state change

```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```

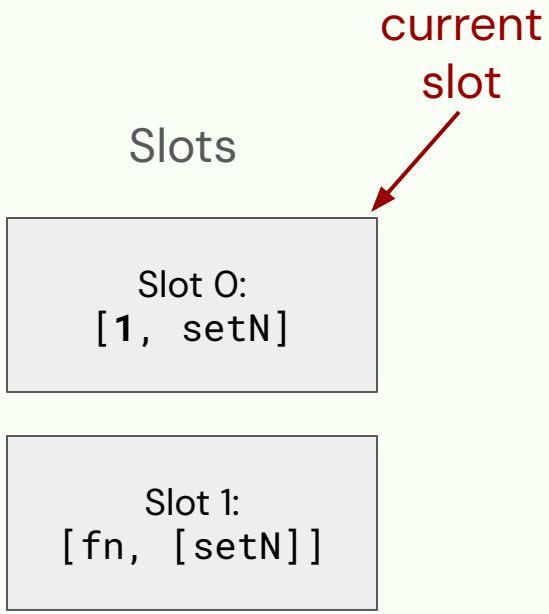
Question: What happens on 2nd rerender?

3 **Re-render** on state change

```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```

This is garbage

This is now 1



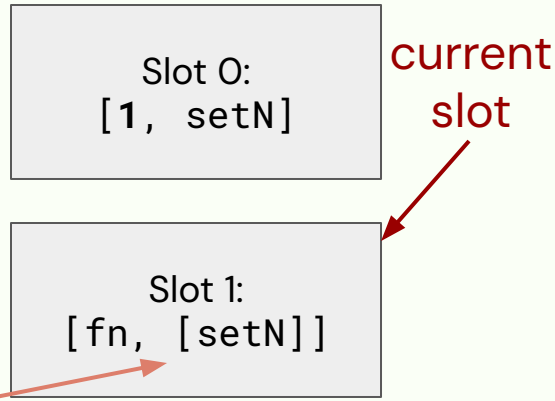
3 **Re-render** on state change

```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```

Deps are compared
(they are equal)

Question: What happens on 2nd rerender?

Slots



3 **Re-render** on state change

```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```

Since deps are equal, the callback is garbage

Question: What happens on 2nd rerender?

Slots

Slot 0:
[~~0~~ → **1**, setN]

Slot 1:
[fn, [setN]]

3 **Re-render** on state change

```
const Counter = () => {
    const [n, setN] = useState(0);
    const onClick = useCallback(() => {
        setN((oldN) => oldN + 1);
    }, [setN]);
    return <button onClick={onClick}>
        Click count: {n}
    </button>;
}
```

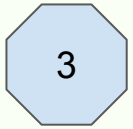
Different VDOM node returned w/ new children

Question: What happens on 2nd rerender?

Slots

Slot 0:
[~~0~~ → **1**, setN]

Slot 1:
[fn, [setN]]



Re-render on state change

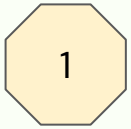
As fast as initial render!

Notable drawback allocated memory can go straight to the garbage collector:

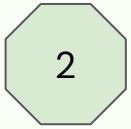
- Hook callbacks
- Hook dependency arrays
- Unchanged (structural) JSX VDOM nodes

Next up: how to compare the two VDOM nodes returned?

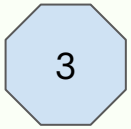
What does React do?

👍 FAST 

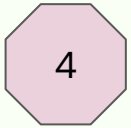
Transform JSX → VDOM

👍 FAST 

Render VDOM → HTML

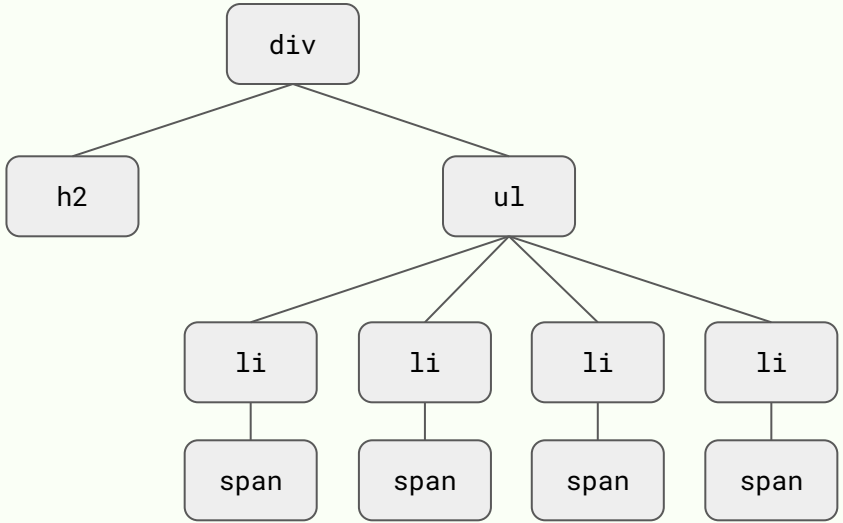
👍 FAST 

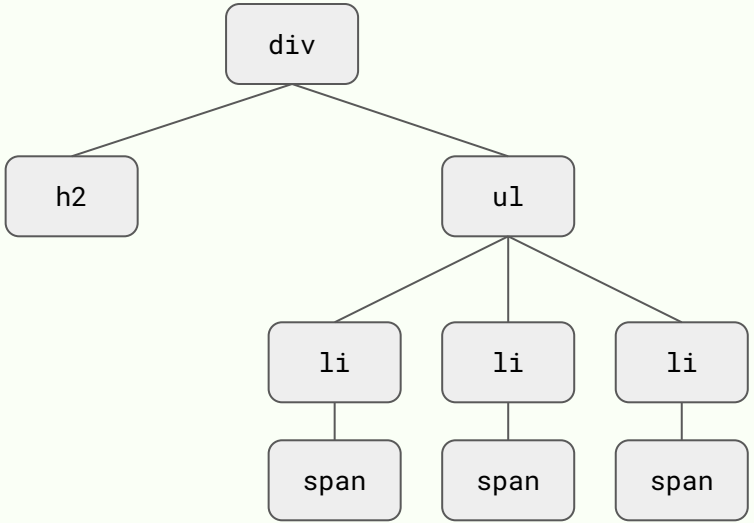
Rerender on state change

🤔 Is this slow? 

Diff $VDOM_a$ ↔ $VDOM_b$
& Apply to HTML

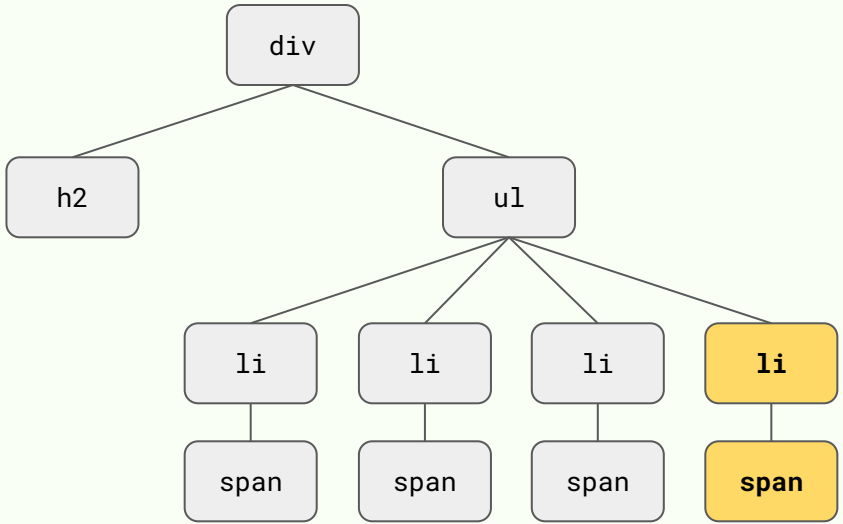
4 **Diff** $VDOM_a \leftrightarrow VDOM_b$ **& Apply** to HTML

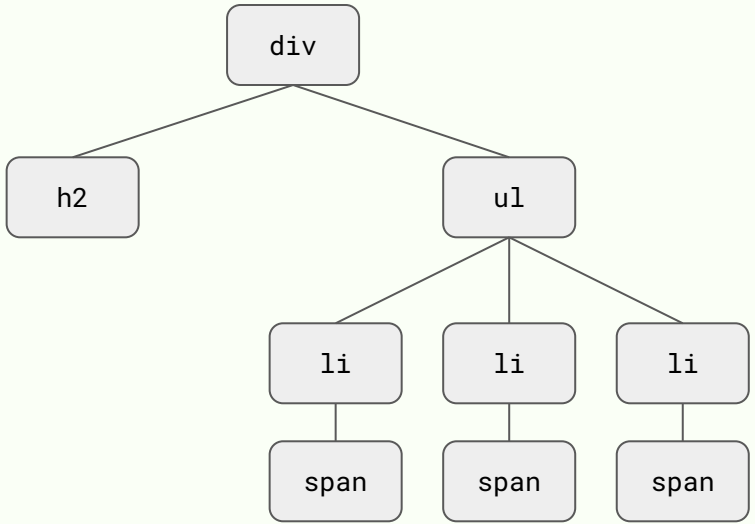




Think like a computer: spot the difference!

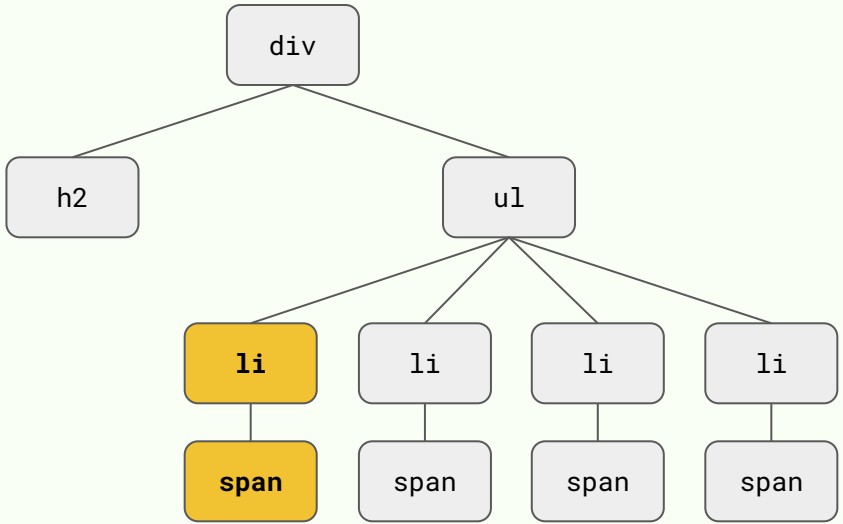
4 **Diff** $VDOM_a$ ⇔ $VDOM_b$ **& Apply** to HTML

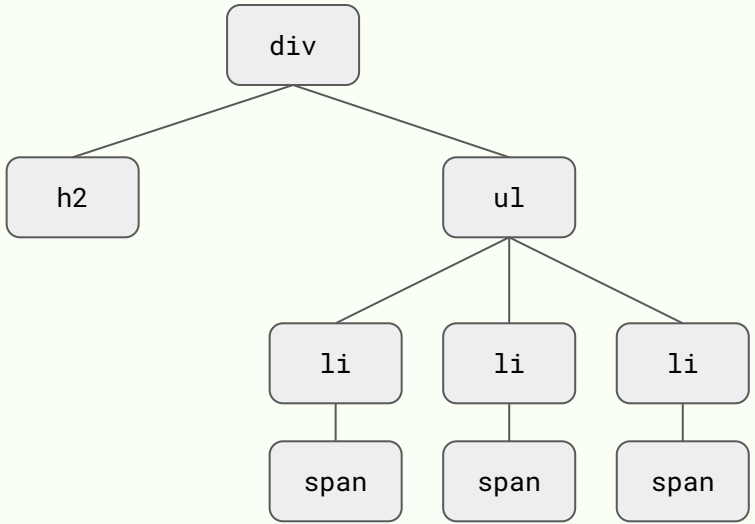




There's an extra subtree on the left!

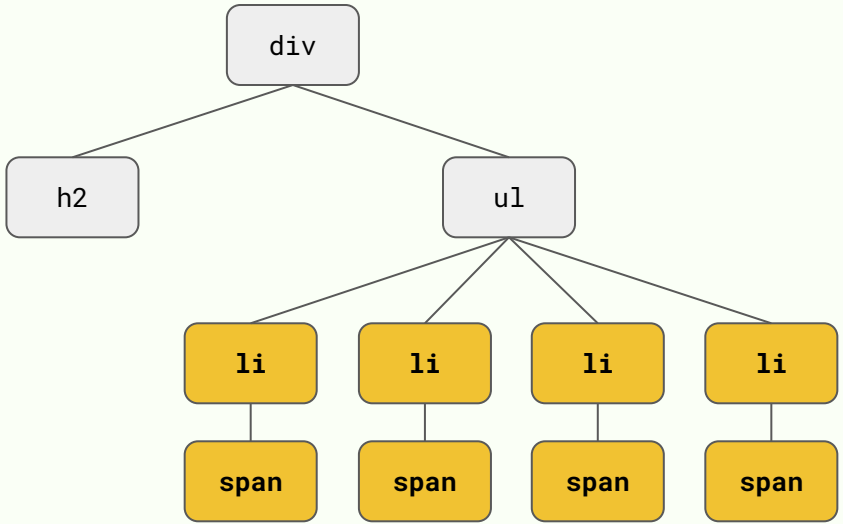
4 **Diff** $VDOM_a \leftrightarrow VDOM_b$ **& Apply** to HTML

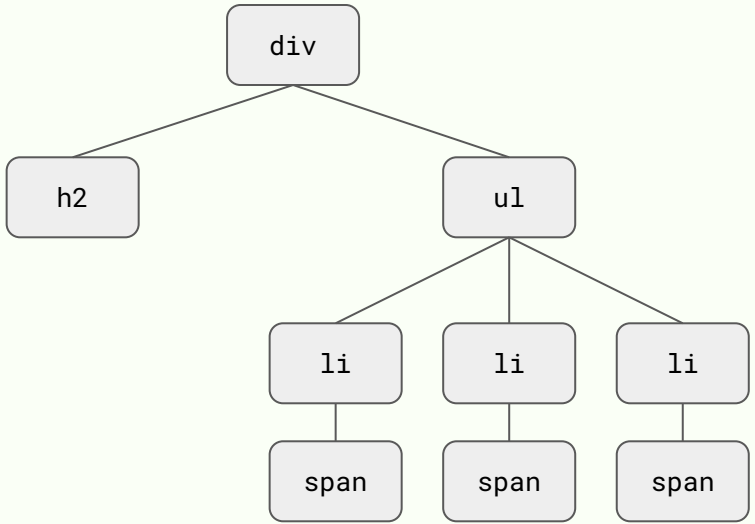




Wait, or is this the extra one?

4 **Diff** $VDOM_a \leftrightarrow VDOM_b$ **& Apply** to HTML

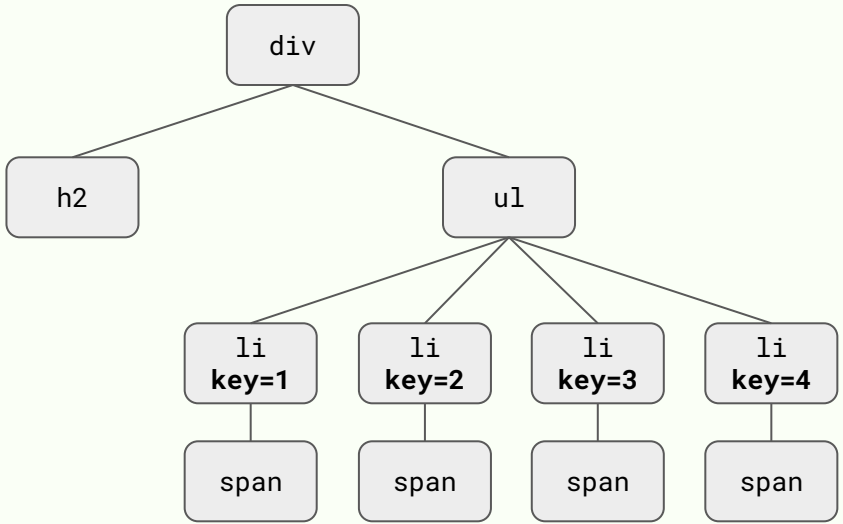


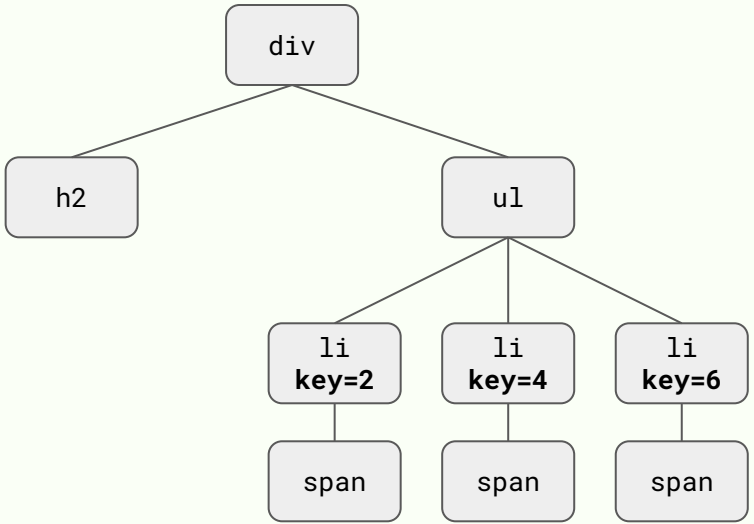


What if they're all different?

Oh no, general tree diffing edit distance is **$O(n^3)$**

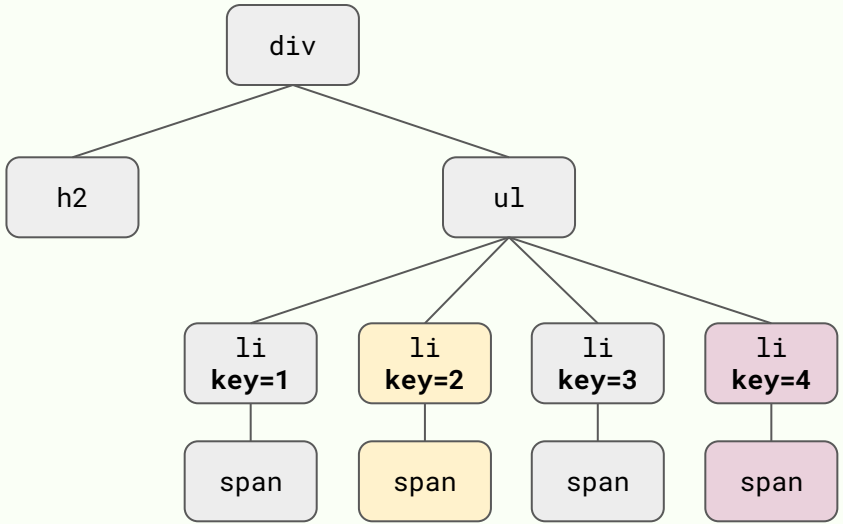
4 **Diff** $VDOM_a \leftrightarrow VDOM_b$ **& Apply** to HTML

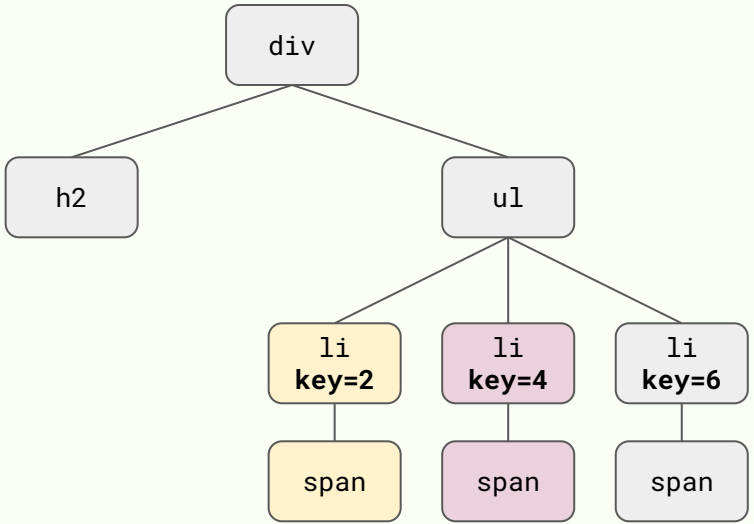




🧐 What if we use a key to tell them apart?

4 **Diff** $VDOM_a \leftrightarrow VDOM_b$ **& Apply** to HTML





The `key` prop exists to turn an **O(n^3)** problem into a **O(n)** problem

4 **Diff** $VDOM_a$ ⇔ $VDOM_b$ **& Apply** to HTML

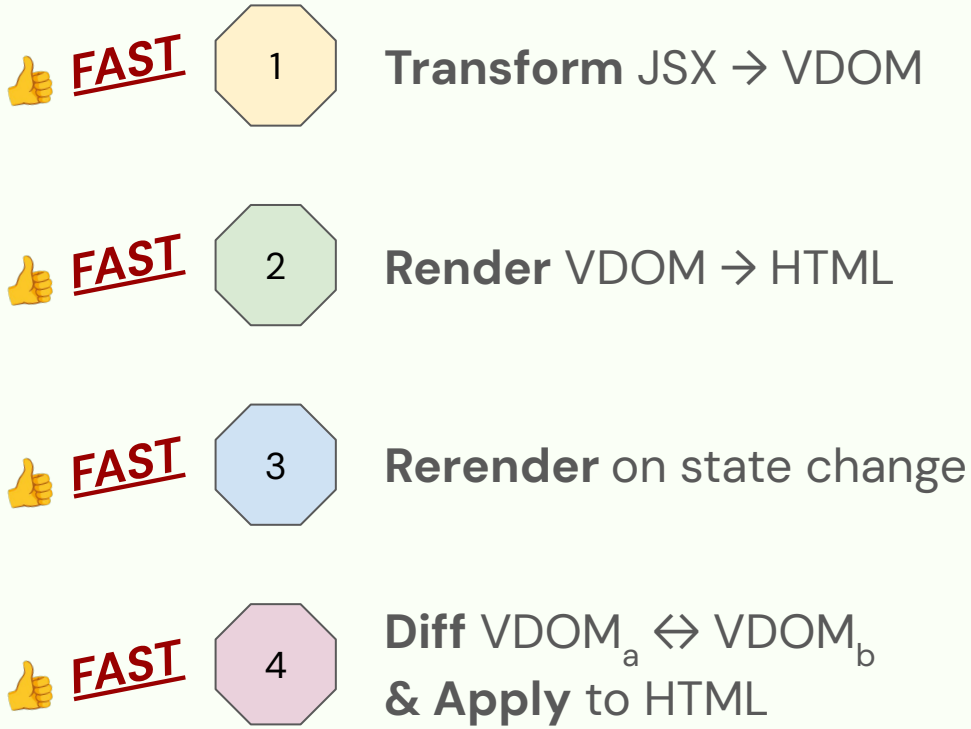
How to diff in React:

- Take a walk over every VDOM node (simultaneously, in both trees)
- Check every prop on every VDOM node
- Use `key` to disambiguate children of the same type
- Skip traversing memoized VDOM ($VDOM_a$ `===` $VDOM_b$)
- When you find differences, apply those edits to HTML

It's fast: ~4-5 linear passes over the number of nodes to perform the diff & apply!

That's way better than **$O(n^3)$**!

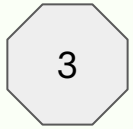
Hey, I thought this talk was called "why React is slow"!



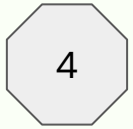
To make it fast,
do less work

To make it small,
do less work

To make it reliable,
do less work



Make the data fit the problem,
not the solution fit the data



Use narrow,
flexible interfaces

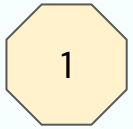
What does ~~React~~ a Frontend Framework ***need*** to do?

1. **Transform** JSX → VDOM
2. **Render** VDOM → HTML
3. **Rerender** on state change
4. **Diff** $VDOM_a$ ↔ $VDOM_b$ **& Apply** to HTML

1. **Transform** JSX → VDOM
2. **Render** VDOM → HTML
3. **Update** HTML when stuff changes

What does Gooey do?

1. **Transform** JSX → VDOM
2. **Render** VDOM → HTML
3. **Update** HTML when stuff changes
4. **Eliminate** all my personal pet peeves with React



Transform JSX → VDOM

Nearly identical to React, nothing fancy here

What does Gooey do?

👍 FAST

1. **Transform** JSX → VDOM
2. **Render** VDOM → HTML
3. **Update** HTML when stuff changes
4. **Eliminate** all my personal pet peeves with React

2 **Render** VDOM → HTML

Gooey has Render-Once™ technology

Components render *exactly* once

No hooks, no rules
(return early, have fun)

Return the JSX you want to see

Bind data to JSX, where you want it

```
const TodoTask: Component<{ name: string }> = ({ name }) => {
 const done = field(false);
 return (
   <li class={calc(() => (done.get() ? 'complete' : ''))}>
     <label>
       <input
         type="checkbox"
         on:click={() => {
           done.set(!done.get());
         }}
         checked={done}
       />
       {calc(() => done.get()
         ? <b>All done!</b>
         : <span>{name}</span>
       )}
     </label>
   </li>
 );
}
```

2 **Render** VDOM → HTML

Gooey has Render-Once™ technology

Components render *exactly* once

No hooks, no rules
(return early, have fun)

Return the JSX you want to see

Bind data to JSX, where you want it

```
const TodoTask: Component<{ name: string }> = ({ name }) => {
 const done = field(false);
 return (
   <li class={calc(() => (done.get() ? 'complete' : ''))}>
     <label>
       <input
         type="checkbox"
         on:click={() => {
           done.set(!done.get());
         }}
         checked={done}
       />
       {calc(() => done.get()
         ? <b>All done!</b>
         : <span>{name}</span>
       )}
     </label>
   </li>
 );
}
```

make local state with `field()`

2 **Render** VDOM → HTML

Gooey has Render-Once™ technology

Components render *exactly* once

No hooks, no rules
(return early, have fun)

Return the JSX you want to see

Bind data to JSX, where you want it

```
const TodoTask: Component<{ name: string }> = ({ name }) => {
 const done = field(false);
 return (
   <li class={calc(() => (done.get() ? 'complete' : ''))}>
     <label>
       <input
         type="checkbox"
         on:click={() => {
           done.set(!done.get());
         }}
         checked={done}
       />
       {calc(() => done.get()
         ? <b>All done!</b>
         : <span>{name}</span>
       )}
     </label>
   </li>
 );
}
```

state objects are getters & setters

2 **Render** VDOM → HTML

Gooey has Render-Once™ technology

Components render *exactly* once

No hooks, no rules
(return early, have fun)

Return the JSX you want to see

Bind data to JSX, where you want it

```
const TodoTask: Component<{ name: string }> = ({ name }) => {
 const done = field(false);
 return (
   <li class={calc(() => (done.get() ? 'complete' : ''))}>
     <label>
       <input
         type="checkbox"
         on:click={() => {
           done.set(!done.get());
         }}
         checked={done}
       />
       {calc(() => done.get()
         ? <b>All done!</b>
         : <span>{name}</span>
       )}
     </label>
   </li>
 );
}
```

put state directly as an attribute on HTML nodes

2 **Render** VDOM → HTML

Gooey has Render-Once™ technology

Components render *exactly* once

No hooks, no rules
(return early, have fun)

Return the JSX you want to see

Bind data to JSX, where you want it

```
const TodoTask: Component<{ name: string }> = ({ name }) => {
 const done = field(false);
 return (
   <li class={calc(() => (done.get() ? 'complete' : ''))}>
     <label>
       <input
         type="checkbox"
         on:click={() => {
           done.set(!done.get());
         }}
         checked={done}
       />
       {calc(() => done.get()
         ? <b>All done!</b>
         : <span>{name}</span>
       )}
     </label>
   </li>
 );
}
```

and use `calc()` for derived state

2 **Render** VDOM → HTML

Gooey has Render-Once™ technology

Components render *exactly* once

No hooks, no rules
(return early, have fun)

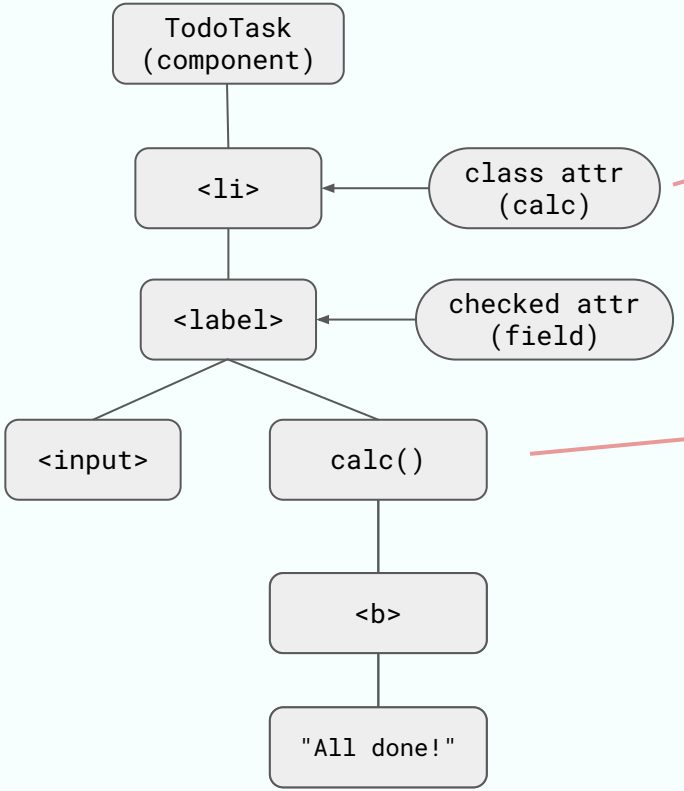
Return the JSX you want to see

Bind data to JSX, where you want it

```
const TodoTask: Component<{ name: string }> = ({ name }) => {
 const done = field(false);
 return (
   <li class={calc(() => (done.get() ? 'complete' : ''))}>
     <label>
       <input
         type="checkbox"
         on:click={() => {
           done.set(!done.get());
         }}
         checked={done}
       />
       {calc(() => done.get()
         ? <b>All done!</b>
         : <span>{name}</span>
       )}
     </label>
   </li>
 );
}
```

put JSX
in a `calc()`
for dynamic
content

2 **Render** VDOM → HTML




```
const TodoTask: Component<{ name: string }> = ({ name }) => {
 const done = field(false);
 return (
   <li class={calc(() => (done.get() ? 'complete' : ''))}>
     <label>
       <input
         type="checkbox"
         on:click={() => {
           done.set(!done.get());
         }}
         checked={done}
       />
       {calc(() => done.get()
         ? <b>All done!</b>
         : <span>{name}</span>
       )}
     </label>
   </li>
 );
}
```

VDOM knows exactly *where* to update when data changes!

What does Gooey do?

👍 FAST

1 **Transform** JSX → VDOM

👍 FAST

2 **Render** VDOM → HTML

3 **Update** HTML when stuff changes

4 **Eliminate** all my personal pet peeves with React

3 **Update** HTML when stuff changes

JSX is either fixed (structural) or dynamic (can be swapped out)

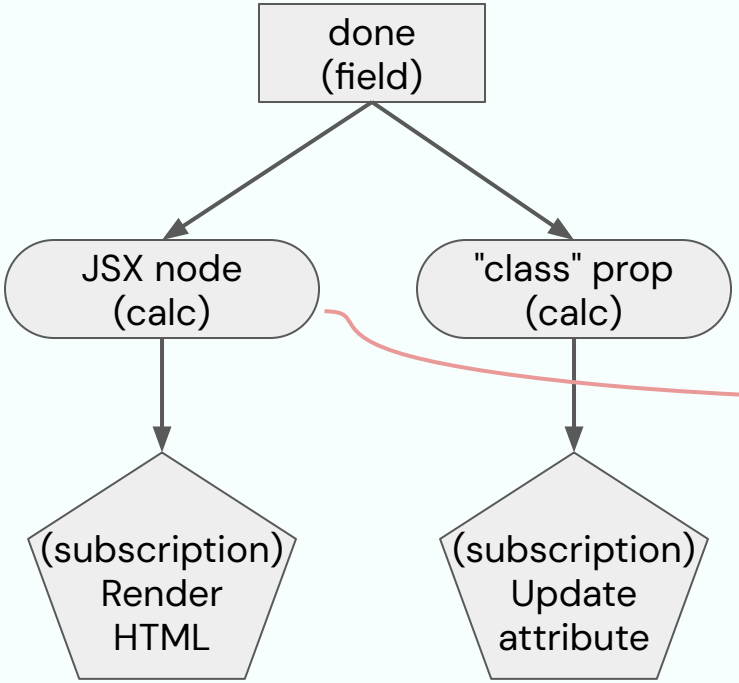
`calc()` automagically* recalculates when state it reads has changed

How does this work?

```
const TodoTask: Component<{ name: string }> = ({ name }) => {
 const done = field(false);
 return (
   <li class={calc(() => (done.get() ? 'complete' : ''))}>
     <label>
       <input
         type="checkbox"
         on:click={() => {
           done.set(!done.get());
         }}
         checked={done}
       />
       {calc(() => done.get()
         ? <b>All done!</b>
         : <span>{name}</span>
       )}
     </label>
   </li>
 );
}
```

*not really magic,
Gooey is a spreadsheet engine

3 Update ~~HTML~~ stuff when stuff changes

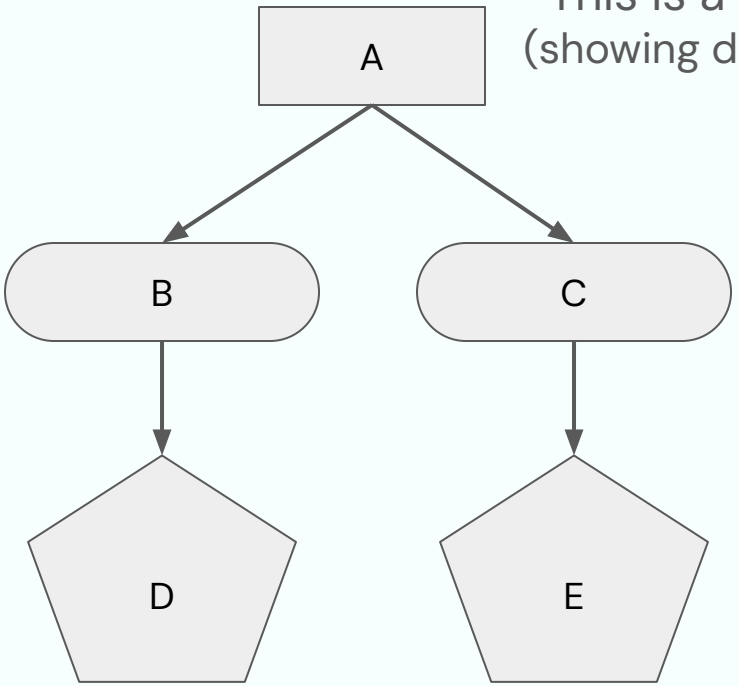



```
const TodoTask: Component<{ name: string }> = ({ name }) => {
 const done = field(false);
 return (
   <li class={calc(() => (done.get() ? 'complete' : ''))}>
     <label>
       <input
         type="checkbox"
         on:click={() => {
           done.set(!done.get());
         }}
         checked={done}
       />
       {calc(() => done.get()
         ? <b>All done!</b>
         : <span>{name}</span>
       )}
     </label>
   </li>
 );
}
```

Note: the dependency graph on the left does not show this "checked" prop binding.

It works the exact same as the "class" prop binding, so I omitted it.

3 **Update** stuff when stuff changes



This is a **Graph**
(showing data flow)

field() = "authoritative state"

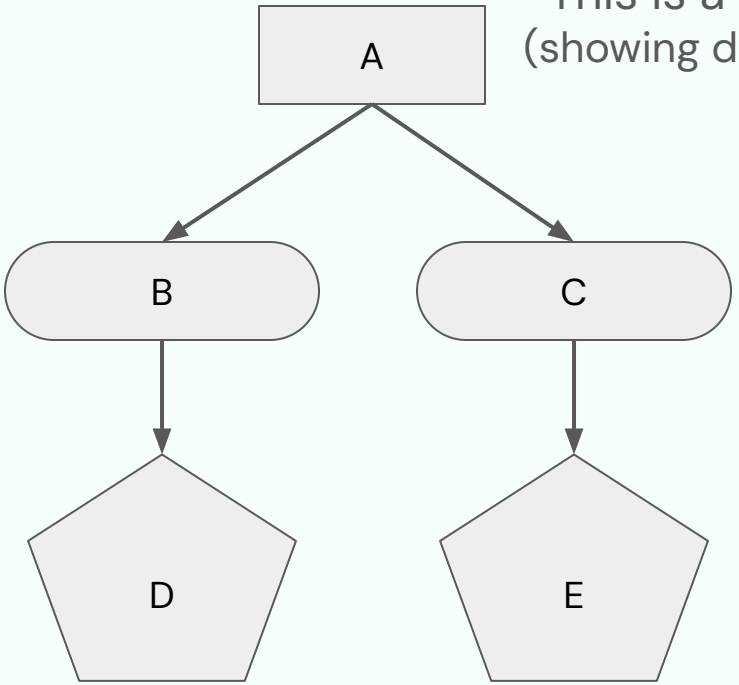
calc() = "derived state" (calc)

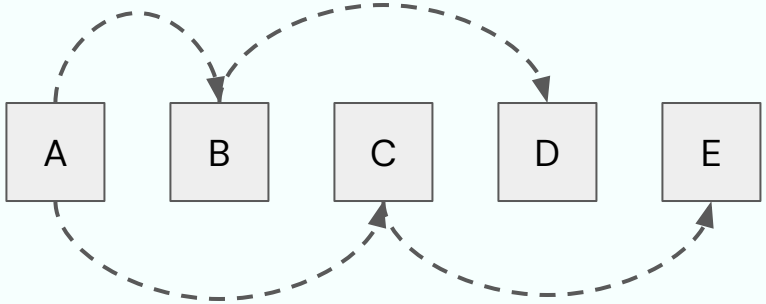
Subscriptions are also nodes

Data flows in arrow direction

3 **Update** stuff when stuff changes

This is a **Graph**
(showing data flow)

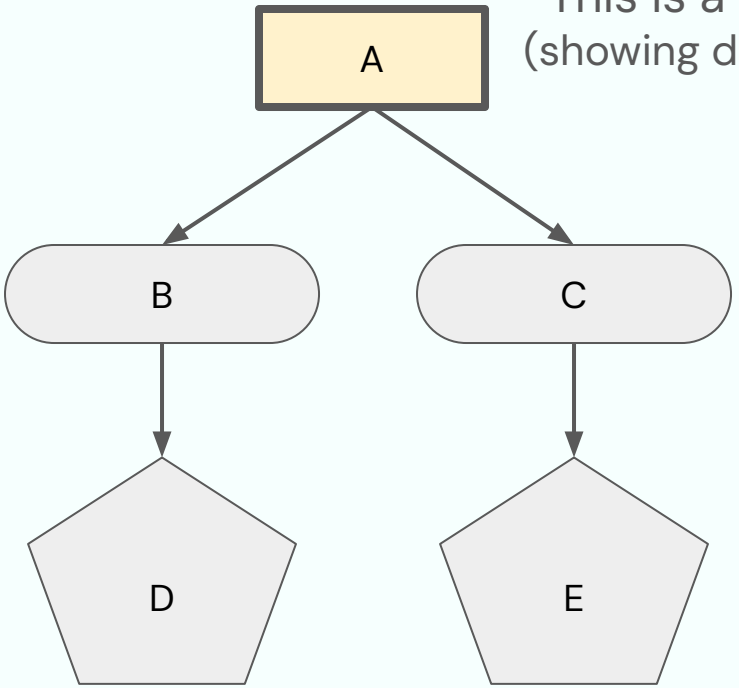


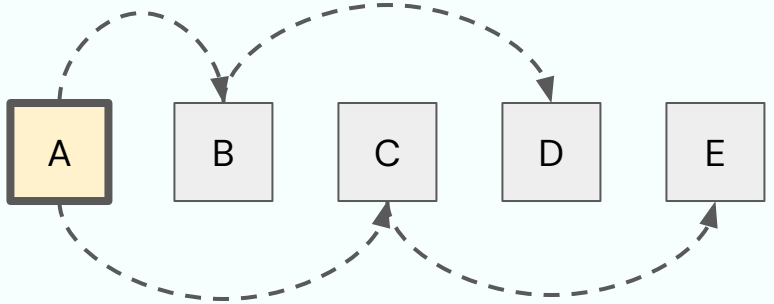


This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes

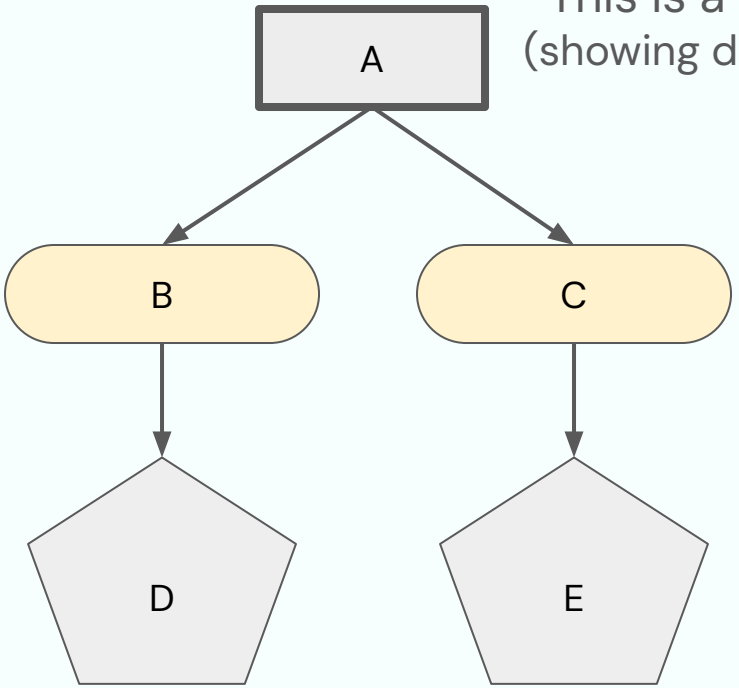
This is a **Graph**
(showing data flow)



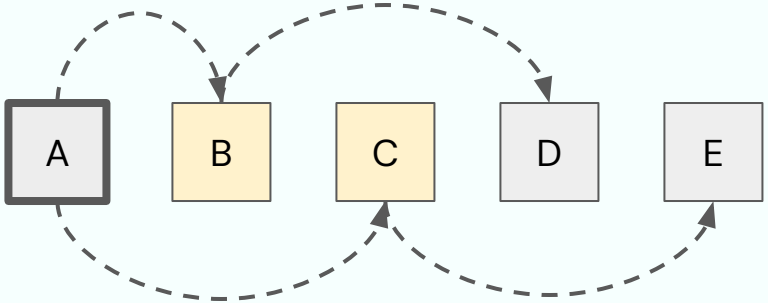


This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes



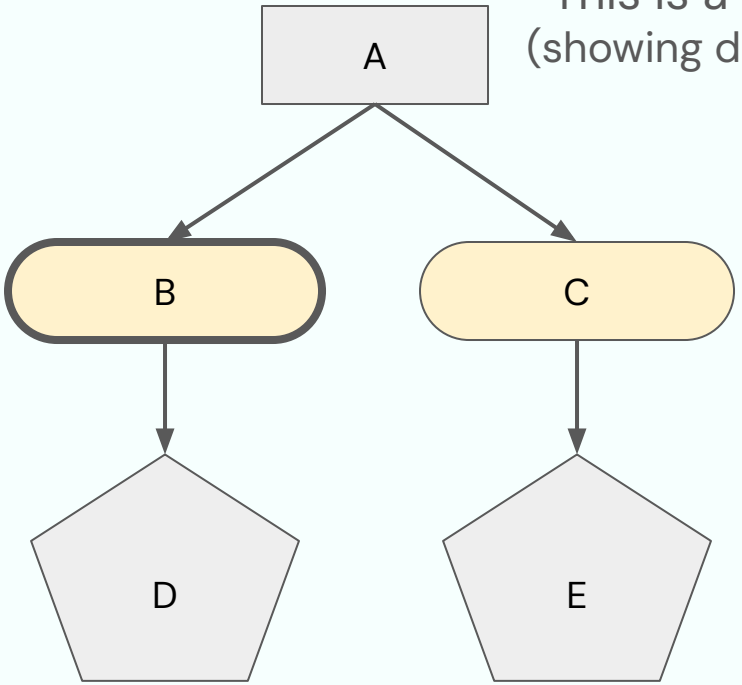
This is a **Graph**
(showing data flow)

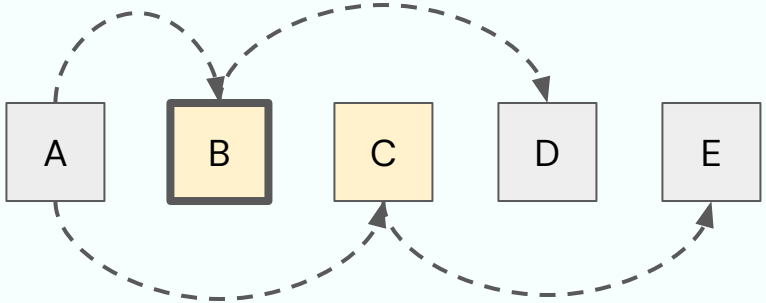


This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes

This is a **Graph**
(showing data flow)

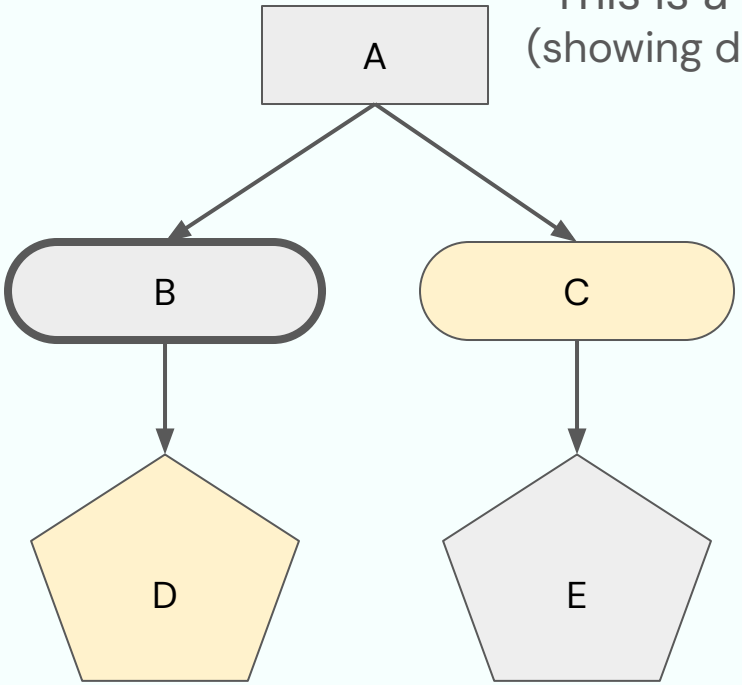


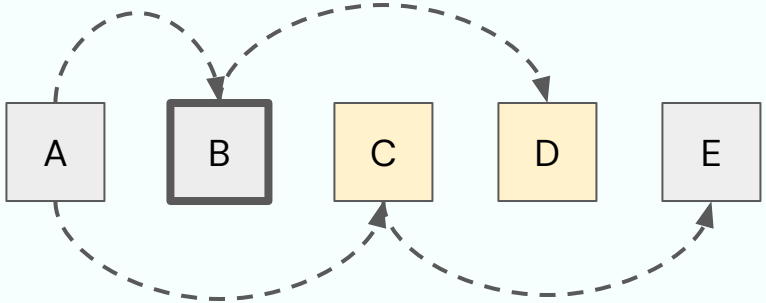


This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes

This is a **Graph**
(showing data flow)

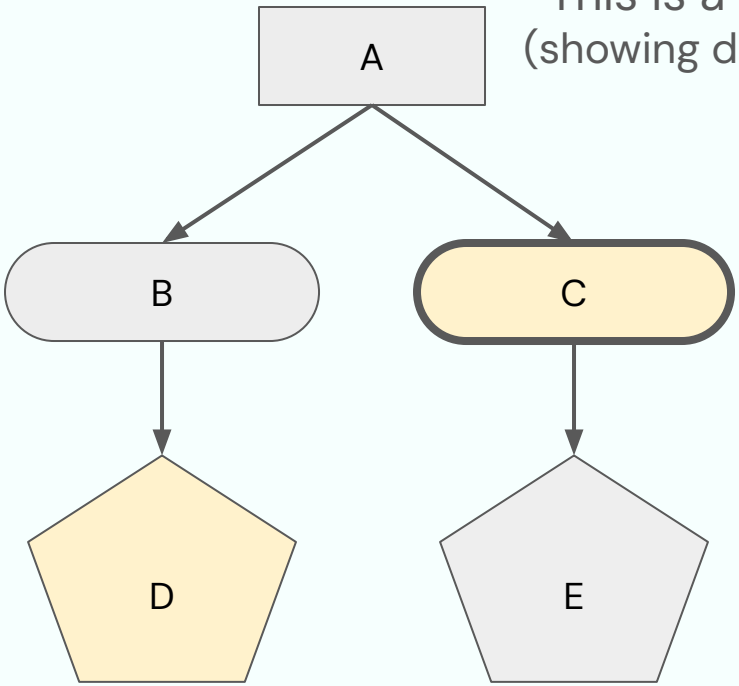


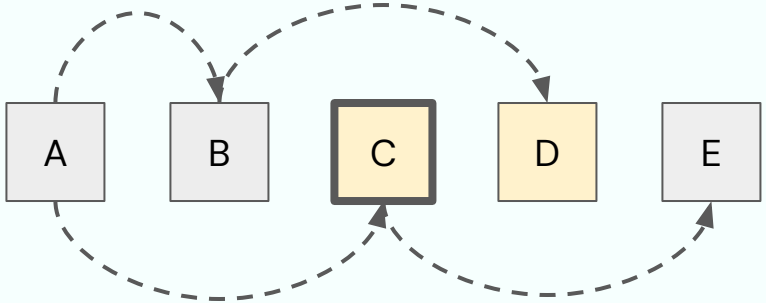


This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes

This is a **Graph**
(showing data flow)

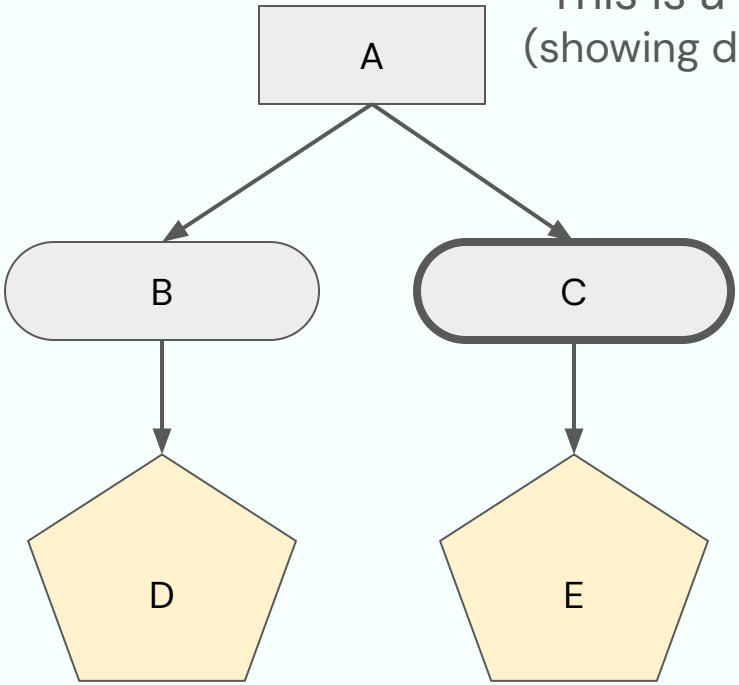


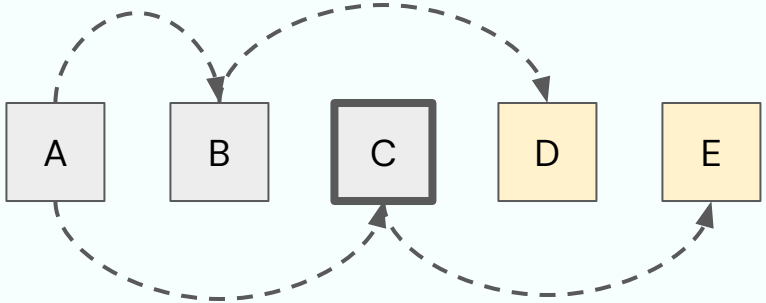


This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes

This is a **Graph**
(showing data flow)

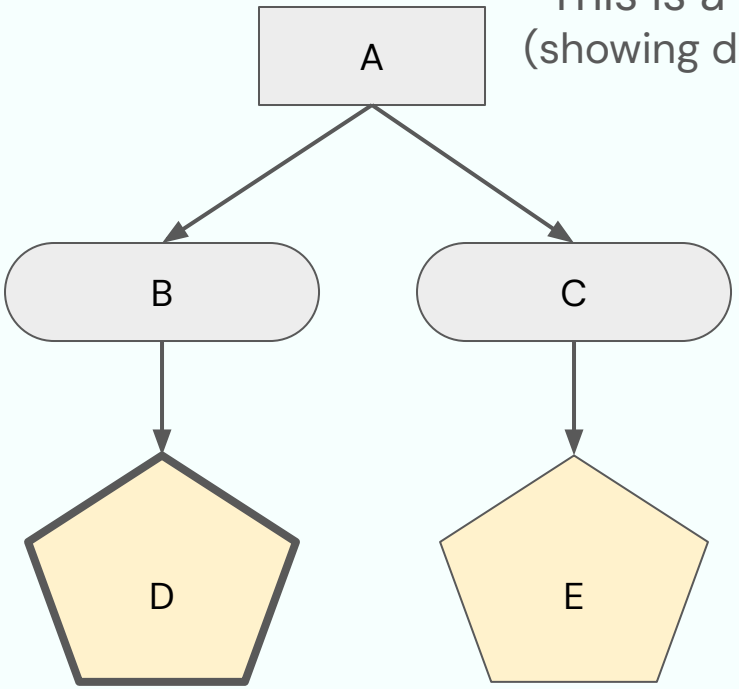


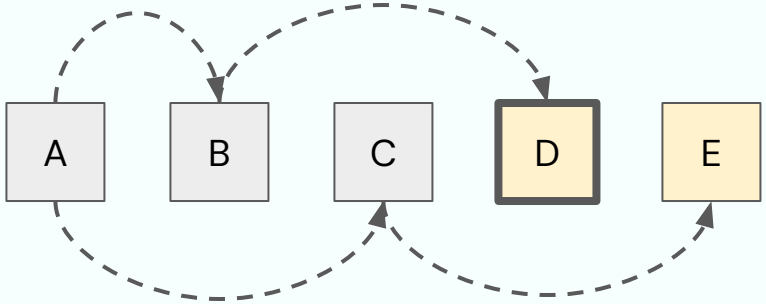


This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes

This is a **Graph**
(showing data flow)

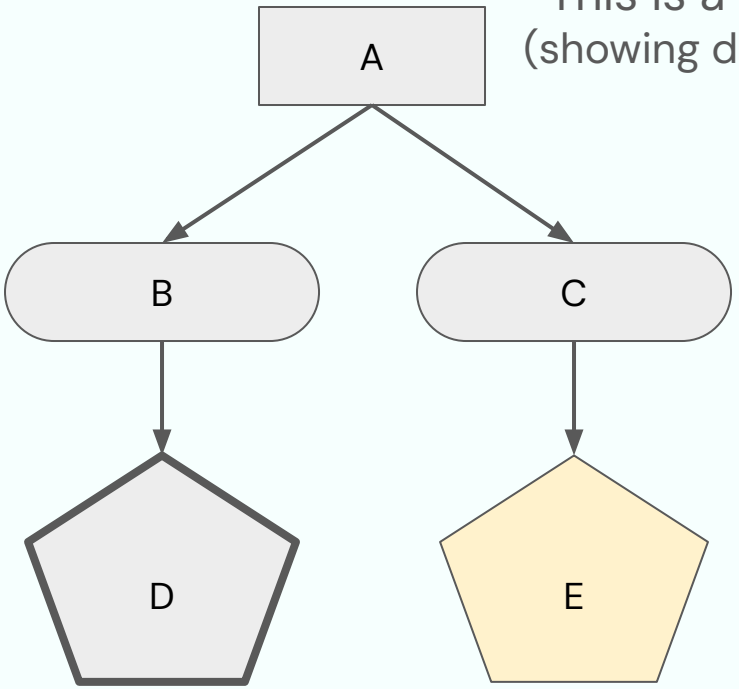


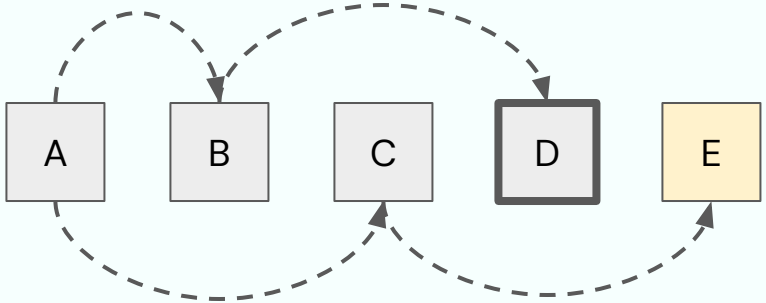


This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes

This is a **Graph**
(showing data flow)

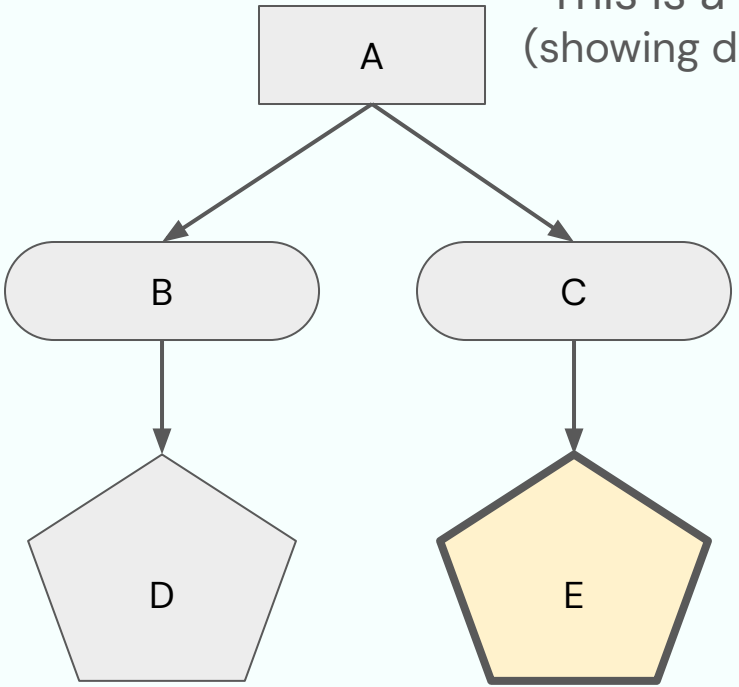


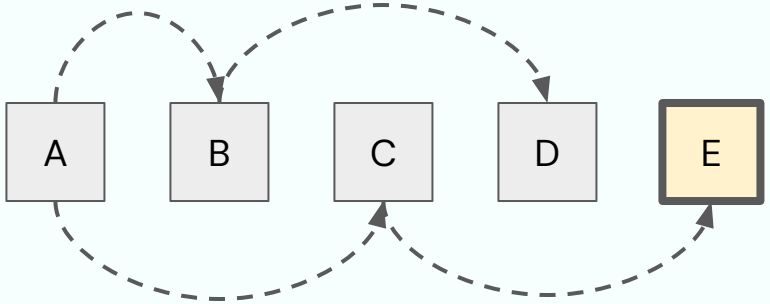


This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes

This is a **Graph**
(showing data flow)

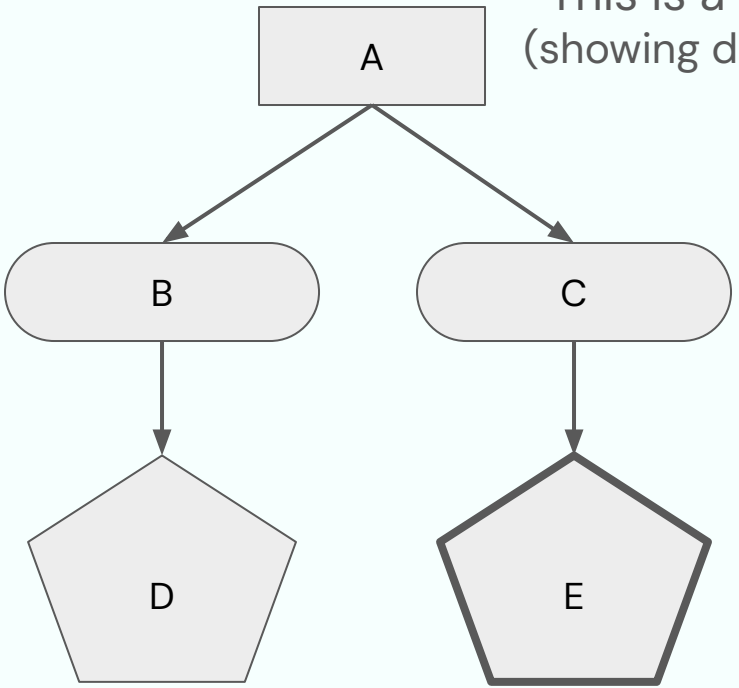


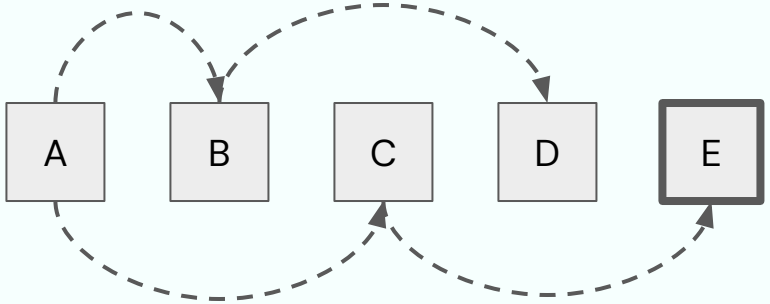


This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes

This is a **Graph**
(showing data flow)

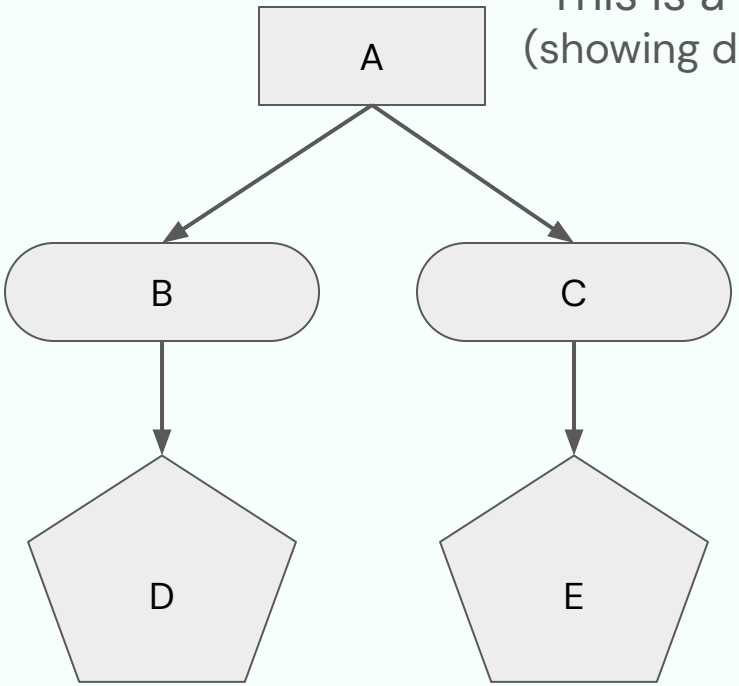


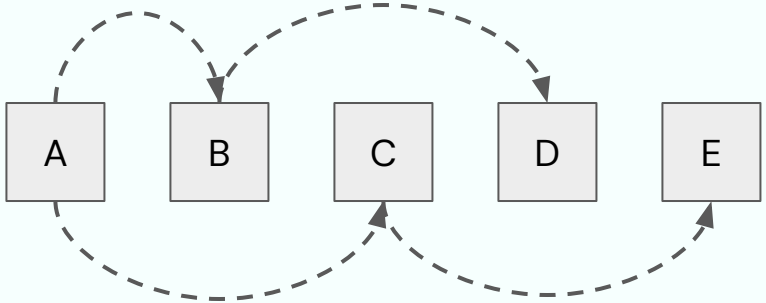


This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes

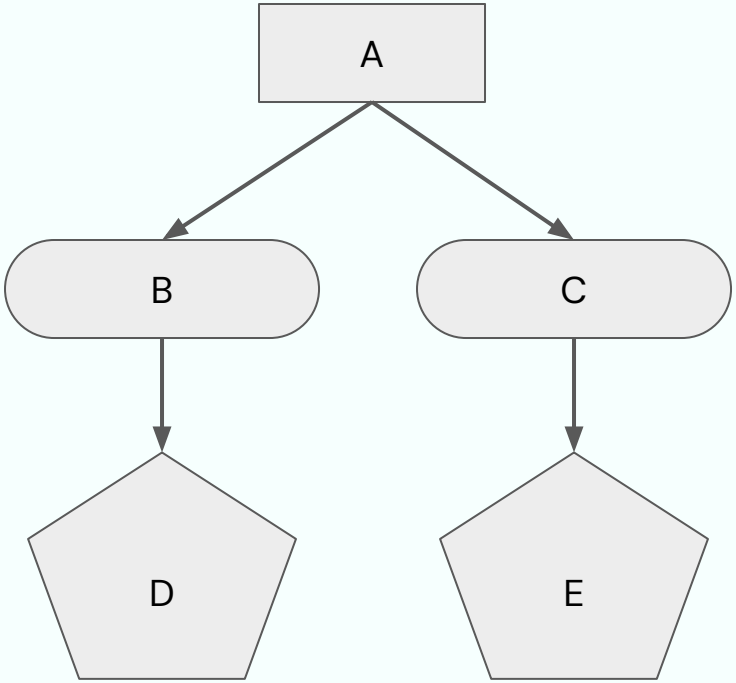
This is a **Graph**
(showing data flow)

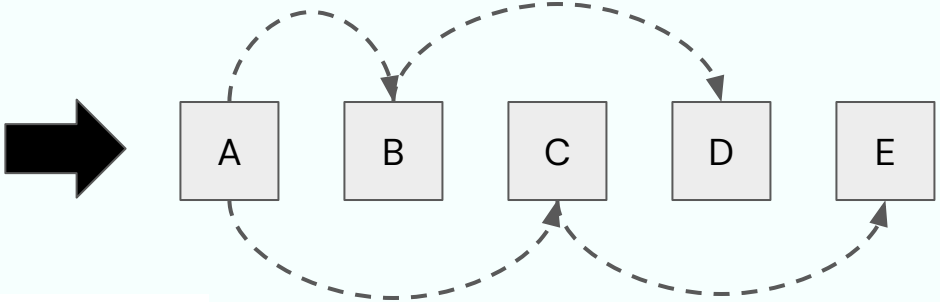




This is a **topological ordering** of the graph
(all the arrows go left-to-right)
(there are many valid topological orderings)

3 **Update** stuff when stuff changes

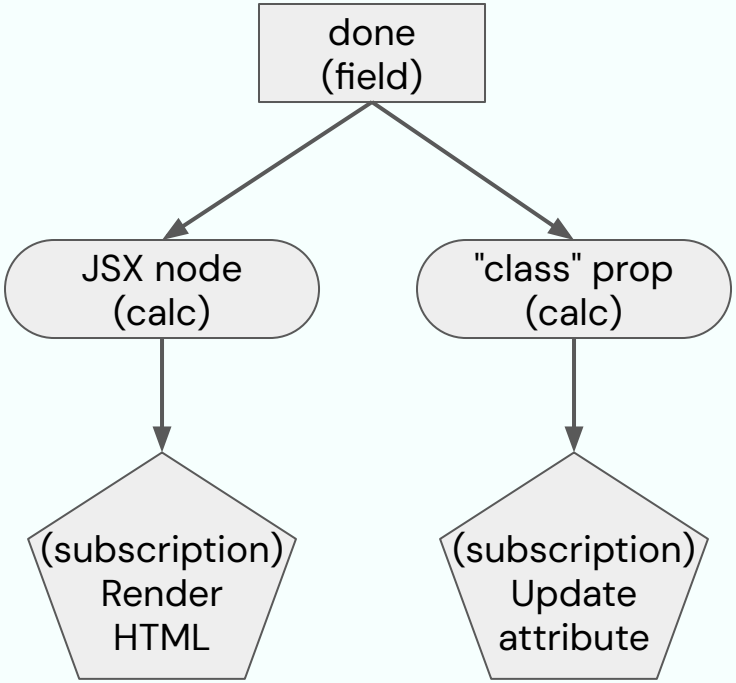






Tarjan's algorithm
Fast: **O(nodes + edges)**
also detects loops for free!
Thanks, Robert Tarjan!

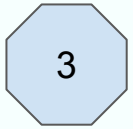
3 **Update** stuff when stuff changes




```
const done = field(false);

const TodoTask: Component<{ name: string }> = ({ name }) => {
 return (
   <li class={calc(() => (done.get() ? 'complete' : ''))}>
     <label>
       <input
         type="checkbox"
         on:click={() => {
           done.set(!done.get());
         }}
         checked={done}
       />
       {calc(() => done.get()
         ? <b>All done!</b>
         : <span>{name}</span>
       )}
     </label>
   </li>
 );
}
```

Global reactive state!



Update stuff when stuff changes

Reactive, fine-grained data is a first-class citizen

Authoritative field() nodes are state

Derived calc() nodes are state

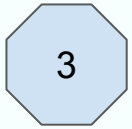
Both can be subscribed to, so they can be placed into JSX as-is

"Re-rendering" happens *inside the JSX*, not as part of the Component interface

It's like a spreadsheet for UI: `field()` are plain cells and `calc()` are formulas

Added bonus: every `calc()` is memoized for free

What about arrays?



Update ~~stuff~~ arrays when ~~stuff~~ arrays change

Collections: special array state

An array of fields & a field length

Fine-grained reactive indexes & length

Derived collections are "mapped views"

Subscribe to (derived) collections

Put (derived) collections in JSX for UI lists

```
const TodoTasks: Component = () => {
  const tasks = collection([
    { name: 'cereal', done: field(false) }
  ]);
  const todoName = field('');
  return (
    <div>
      <ul>{tasks.mapView(
        (task) => <TodoTask task={task} />
      )}</ul>
      <form on:submit={(e, el) => {
        el.preventDefault();
        tasks.push({
          name: todoName.get(),
          done: field(false),
        });
        todoName.set('');
      }}>
        <input
          type="text"
          value={todoName}
          on:input={(e, el) => todoName.set(el.value)}
        />
        <button type="submit">Add Todo</button>
      </form>
    </div>
  );
}
```

3 Update ~~stuff~~ arrays when ~~stuff~~ arrays change

Collections: special array state

An array of fields & a field length

Fine-grained reactive indexes & length

Derived collections are "mapped views"

Subscribe to (derived) collections

Put (derived) collections in JSX for UI lists

```
const TodoTasks: Component = () => {
  const tasks = collection([
    { name: 'cereal', done: field(false) }
  ]);
  const todoName = field('');
  return (
    <div>
      <ul>{tasks.mapView(
        (task) => <TodoTask task={task} />
      )}</ul>
      <form on:submit={(e, el) => {
        el.preventDefault();
        tasks.push({
          name: todoName.get(),
          done: field(false),
        });
        todoName.set('');
      }}>
        <input
          type="text"
          value={todoName}
          on:input={(e, el) => todoName.set(el.value)}
        />
        <button type="submit">Add Todo</button>
      </form>
    </div>
  );
}
```

Make a collection

3 Update ~~stuff~~ arrays when ~~stuff~~ arrays change

Collections: special array state

An array of fields & a field length

Fine-grained reactive indexes & length

Derived collections are "mapped views"

Subscribe to (derived) collections

Put (derived) collections in JSX for UI lists

```
const TodoTasks: Component = () => {
  const tasks = collection([
    { name: 'cereal', done: field(false) }
  ]);
  const todoName = field('');
  return (
    <div>
      <ul>{tasks.mapView(
        (task) => <TodoTask task={task} />
      )}</ul>
      <form on:submit={(e, el) => {
        el.preventDefault();
        tasks.push({
          name: todoName.get(),
          done: field(false),
        });
        todoName.set('');
      }}>
        <input
          type="text"
          value={todoName}
          on:input={(e, el) => todoName.set(el.value)}
        />
        <button type="submit">Add Todo</button>
      </form>
    </div>
  );
}
```

Mutable like an array

3 Update ~~stuff~~ arrays when ~~stuff~~ arrays change

Collections: special array state

An array of fields & a field length

Fine-grained reactive indexes & length

Derived collections are "mapped views"

Subscribe to (derived) collections

Put (derived) collections in JSX for UI lists

```
const TodoTasks: Component = () => {
  const tasks = collection([
    { name: 'cereal', done: field(false) }
  ]);
  const todoName = field('');
  return (
    <div>
      <ul>{tasks.mapView(
        (task) => <TodoTask task={task} />
      )}</ul>
      <form on:submit={(e, el) => {
        el.preventDefault();
        tasks.push({
          name: todoName.get(),
          done: field(false),
        });
        todoName.set('');
      }}>
        <input
          type="text"
          value={todoName}
          on:input={(e, el) => todoName.set(el.value)}
        />
        <button type="submit">Add Todo</button>
      </form>
    </div>
  );
}
```

"Project"/map to a derived collection of JSX nodes

3 Update ~~stuff~~ arrays when ~~stuff~~ arrays change

Collections: special array state

An array of fields & a field length

Fine-grained reactive indexes & length

Derived collections are "mapped views"

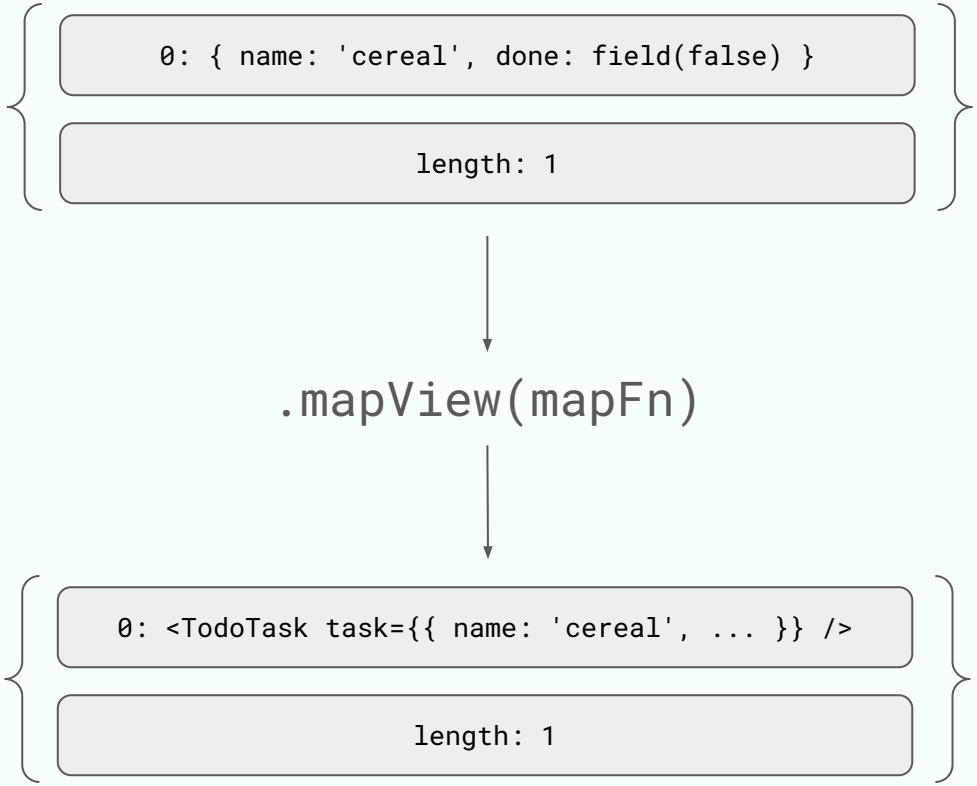
Subscribe to (derived) collections

Put (derived) collections in JSX for UI lists

```
const TodoTasks: Component = () => {
  const tasks = collection([
    { name: 'cereal', done: field(false) }
  ]);
  const todoName = field('');
  return (
    <div>
      <ul>{tasks.mapView(
        (task) => <TodoTask task={task} />
      )}</ul>
      <form on:submit={(e, el) => {
        el.preventDefault();
        tasks.push({
          name: todoName.get(),
          done: field(false),
        });
        todoName.set('');
      }}>
        <input
          type="text"
          value={todoName}
          on:input={(e, el) => todoName.set(el.value)}
        />
        <button type="submit">Add Todo</button>
      </form>
    </div>
  );
}
```

No key prop needed, it Just Works™

3 Update ~~stuff~~ arrays when ~~stuff~~ arrays change




```
const TodoTasks: Component = () => {
  const tasks = collection([
    { name: 'cereal', done: field(false) }
  ]);
  const todoName = field('');
  return (
    <div>
      <ul>{tasks.mapView(
        (task) => <TodoTask task={task} />
      )}</ul>
      <form on:submit={(e, el) => {
        el.preventDefault();
        tasks.push({
          name: todoName.get(),
          done: field(false),
        });
        todoName.set('');
      }}>
        <input
          type="text"
          value={todoName}
          on:input={(e, el) => todoName.set(el.value)}
        />
        <button type="submit">Add Todo</button>
      </form>
    </div>
  );
}
```

3 Update ~~stuff~~ arrays when ~~stuff~~ arrays change

```
0: { name: 'cereal', done: field(false) }
1: { name: 'milk', done: field(false) }
length: 2
```

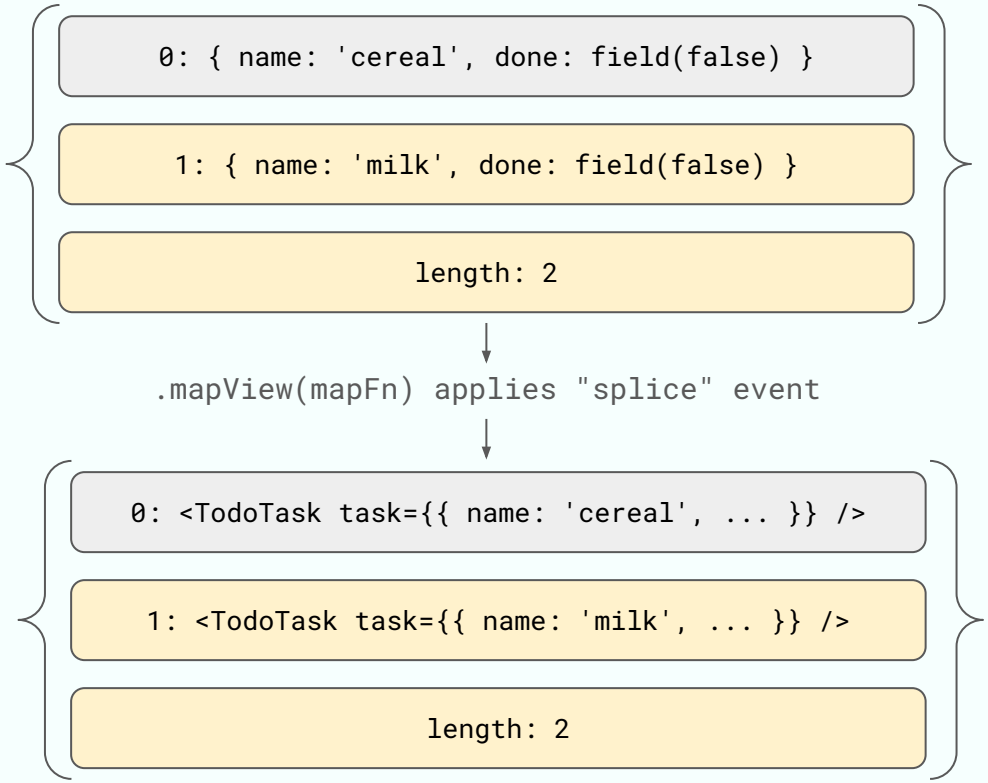
.push(. . .) emits collection event

```
"splice" event
{ index: 1, items: [{ name: 'milk', ... }] }
```

```
const TodoTasks: Component = () => {
  const tasks = collection([
    { name: 'cereal', done: field(false) }
  ]);
  const todoName = field('');
  return (
    <div>
      <ul>{tasks.mapView(
        (task) => <TodoTask task={task} />
      )}</ul>
      <form on:submit={(e, el) => {
        el.preventDefault();
        tasks.push({
          name: todoName.get(),
          done: field(false),
        });
        todoName.set('');
      }}>
        <input
          type="text"
          value={todoName}
          on:input={(e, el) => todoName.set(el.value)}
        />
        <button type="submit">Add Todo</button>
      </form>
    </div>
  );
}
```

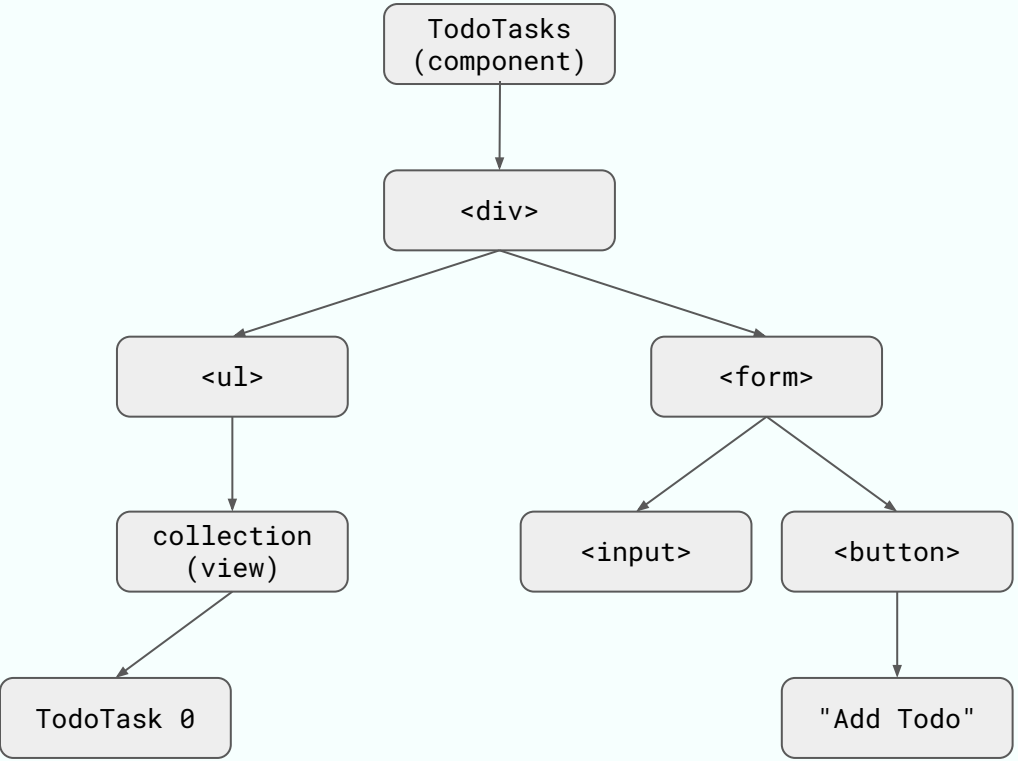
User adds a new todo item

3 Update ~~stuff~~ arrays when ~~stuff~~ arrays change



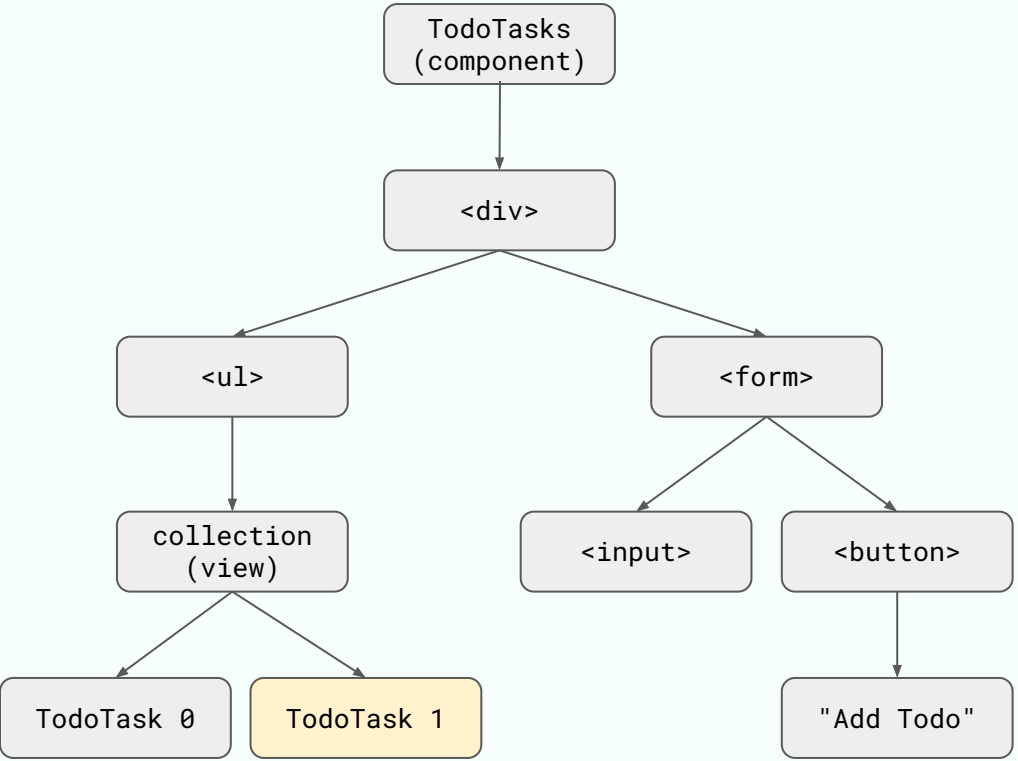

```
const TodoTasks: Component = () => {
  const tasks = collection([
    { name: 'cereal', done: field(false) }
  ]);
  const todoName = field('');
  return (
    <div>
      <ul>{tasks.mapView(
        (task) => <TodoTask task={task} />
      )}</ul>
      <form on:submit={(e, el) => {
        el.preventDefault();
        tasks.push({
          name: todoName.get(),
          done: field(false),
        });
        todoName.set('');
      }}>
        <input
          type="text"
          value={todoName}
          on:input={(e, el) => todoName.set(el.value)}
        />
        <button type="submit">Add Todo</button>
      </form>
    </div>
  );
}
```

3 Update ~~stuff~~ arrays when ~~stuff~~ arrays change



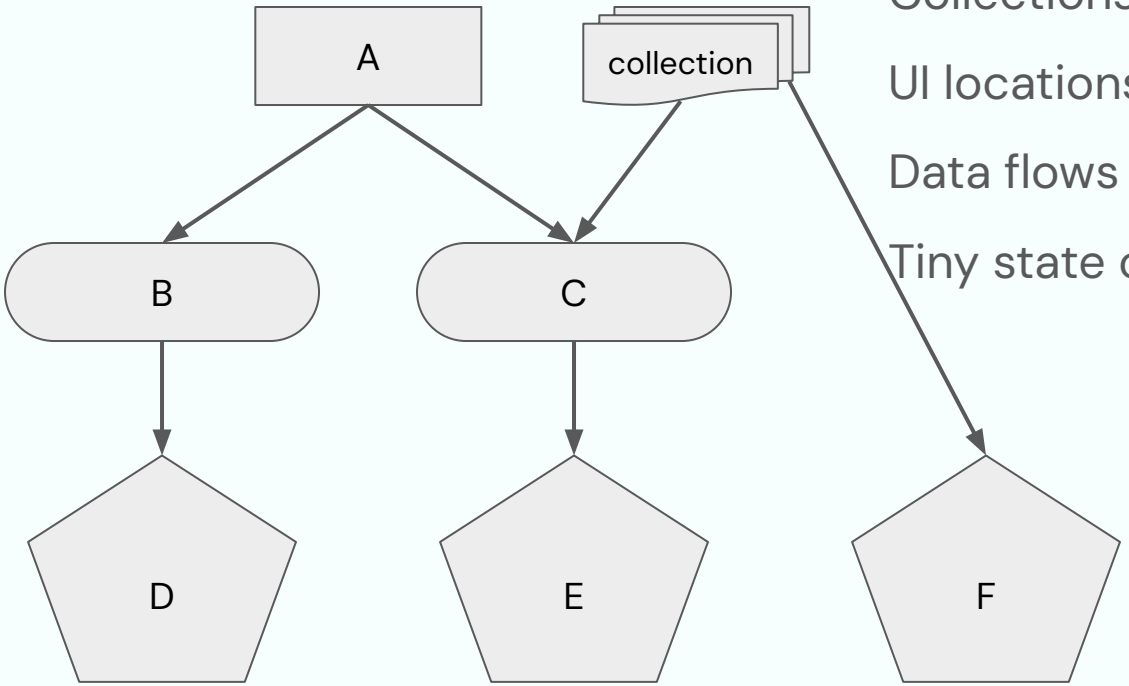

```
const TodoTasks: Component = () => {
  const tasks = collection([
    { name: 'cereal', done: field(false) }
  ]);
  const todoName = field('');
  return (
    <div>
      <ul>{tasks.mapView(
        (task) => <TodoTask task={task} />
      )}</ul>
      <form on:submit={(e, el) => {
        el.preventDefault();
        tasks.push({
          name: todoName.get(),
          done: field(false),
        });
        todoName.set('');
      }}>
        <input
          type="text"
          value={todoName}
          on:input={(e, el) => todoName.set(el.value)}
        />
        <button type="submit">Add Todo</button>
      </form>
    </div>
  );
}
```

3 Update ~~stuff~~ arrays when ~~stuff~~ arrays change




```
const TodoTasks: Component = () => {
  const tasks = collection([
    { name: 'cereal', done: field(false) }
  ]);
  const todoName = field('');
  return (
    <div>
      <ul>{tasks.mapView(
        (task) => <TodoTask task={task} />
      )}</ul>
      <form on:submit={(e, el) => {
        el.preventDefault();
        tasks.push({
          name: todoName.get(),
          done: field(false),
        });
        todoName.set('');
      }}>
        <input
          type="text"
          value={todoName}
          on:input={(e, el) => todoName.set(el.value)}
        />
        <button type="submit">Add Todo</button>
      </form>
    </div>
  );
}
```

3 **Update** stuff when stuff changes



Collections in the dependency graph

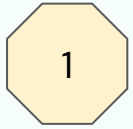
UI locations in the VDOM

Data flows into UI only when needed

Tiny state changes to big apps are fast!

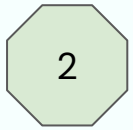
What does Gooey do?

👍 FAST



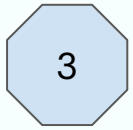
Transform JSX → VDOM

👍 FAST

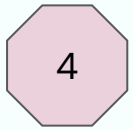


Render VDOM → HTML

👍 FAST

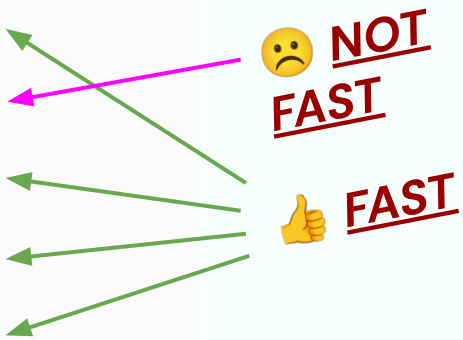


Update HTML when stuff changes



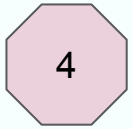
Eliminate all my personal pet peeves with React

	React	Gooey	Comparison factor
Add 10k items	206.85ms (48.34 items/ms)	91.50ms (109.29 items/ms)	**Gooey is 2.26x faster**
Update 10k items	11.55ms (865.80 items/ms)	77.05ms (129.79 items/ms)	**React is 6.67x faster**
Update 10 random items	3.80ms (2.63 items/ms)	0.50ms (20.00 items/ms)	**Gooey is 7.60x faster**
Insert 10 random items	4.85ms (2.06 items/ms)	3.10ms (3.23 items/ms)	**Gooey is 1.57x faster**
Delete 10 random items	4.80ms (2.08 items/ms)	3.10ms (3.23 items/ms)	**Gooey is 1.55x faster**
Clear 10k items	34.40ms (290.70 items/ms)	45.70ms (218.82 items/ms)	**React is 1.33x faster**
Gooey v0.22.0 vs React 19.1.0; Median run over 100 runs on Chrome 135.0.7049.96 Benchmark performed on a MacBook Pro (16-inch, 2019)			



React is fast at updates, but slow at initial rendering. It really suffers when applications get large and small state changes cause big re-renders.

Gooey is designed to be very fast at initial rendering and fast to apply precision updates to small state changes, which are the most common places where speed really matters.



Eliminate all my personal pet peeves with React

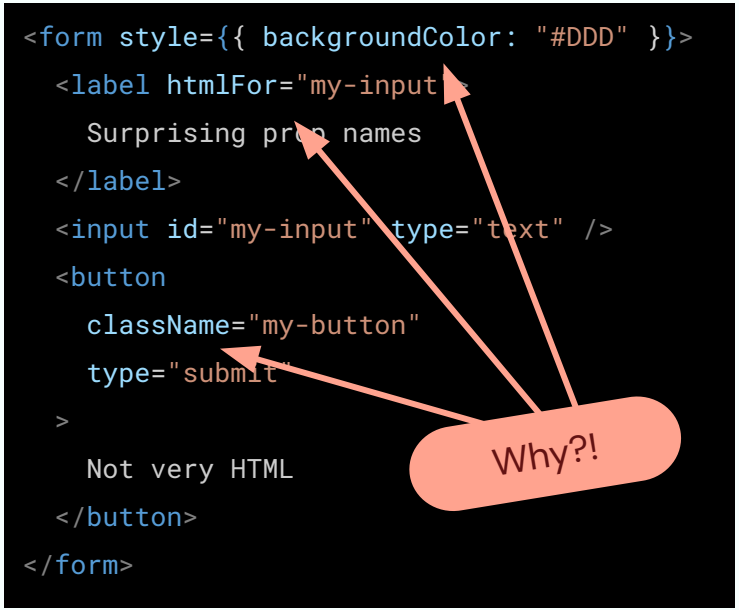
Just for fun, let's see what other tricks Gooey can do

4 **Eliminate** all my personal pet peeves with React

Gooey JSX is like HTML

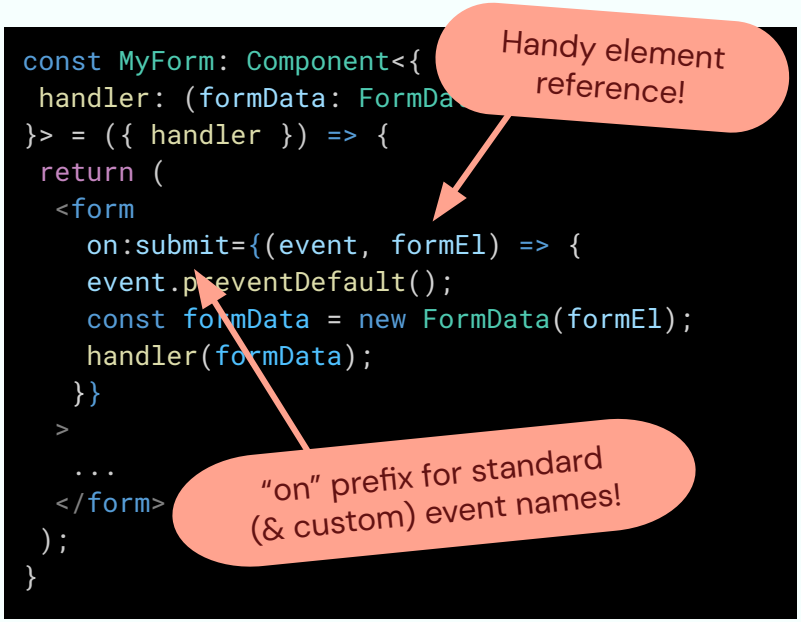
```
<form style="background-color: #DDD">
  <label for="my-input">
    Looks like HTML
  </label>
  <input id="my-input" type="text" />
  <button
    class="my-button"
    type="submit"
  >
    Acts like HTML
  </button>
</form>
```

React JSX is not like HTML



4 **Eliminate** all my personal pet peeves with React

Gooey events are helpful




```
const MyForm: Component<{
 handler: (formData: FormDa
}> = ({ handler }) => {
 return (
  <form
    on:submit={(event, formEl) => {
    event.preventDefault();
    const formData = new FormData(formEl);
    handler(formData);
   }}
  >
   ...
  </form>
 );
}
```

React events are synthetic

```
const MyForm = ({ handler }: {
 handler: (formData: FormData) => void
}) => {
 return (
  <form
   onSubmit={(event) => {
    event.preventDefault();
    const formData = new FormData(
     event.target
    );
    handler(formData);
   }}
  >
   ...
  </form>
 );
}
```

4 **Eliminate** all my personal pet peeves with React

Gooey style is like HTML, and nice

```
<>
 <span style="color: red">
  A plain style string
 </span>
 <span style:color="red">
  Or bind to a CSS property
 </span>
</>
```

Bind value directly to a style!

React style is an object?

```
<>
 <span style={{ color: "red" }}>
  Must be an object
 </span>
</>
```

4 **Eliminate** all my personal pet peeves with React

Gooey JSX custom elements

```
<>
 <span cssprop:duration="50ms">
  Equivalent to:
  el.style.setProperty('--duration, '50ms');
 </span>
 <x-el prop:myProp="cool">
  Equivalent to:
  el.myProp = 'cool';
 </x-el>
 <x-el attr:myAttr="nice">
  Equivalent to:
  el.setAttribute('myAttr', 'yes');
 </x-el>
</>
```

React JSX is stuck in 2013

```
[Object object]
```

Set CSS custom properties!

Works with custom elements!

4 **Eliminate** all my personal pet peeves with React

Gooey components are helpful

```
type State =
 | { type: 'loading' }
 | { type: 'error' }
 | { type: 'ready'; text: string };
const TextGrabber: Component<{ url: string }> = (
 { url },
 { onMount }
) => {
 const state = field<State>({ type:
 onMount(() => {
  fetch(url)
   .then((response) => response.text())
   .then((text) => state.set({ type: 'ready', text }))
   .catch(() => state.set({ type: 'error' }));
 });
 return (
  <span class="TextGrabber">
   Text:
   {calc(() => {
    const result = state.get();
    if (result.type === 'loading') {
     return <span class="loading">...</span>;
    } else if (result.type === 'error') {
     return <span class="error">Error!</span>;
    } else {
     return <span class="success">{result.text}</span>;
    }
   })}
  </span>
 );
};
```

Called **exactly** once per use

Clear lifecycle method names

React hooks sure are something!

```
type State =
 | { type: 'loading' }
 | { type: 'error' }
 | { type: 'ready'; text: string };
const TextGrabber = ({
 const [st, setState]
  type: 'loading'
 });
 useEffect(() => {
  fetch(url)
   .then((response) => response.text())
   .then((text) => setState({ type: 'ready', text }))
   .catch(() => setState({ type: 'error' }));
 }, [setState]);
 return (
  <span className="TextGrabber">
   Text:
   {st.type === 'loading' &&
    <span className="loading">...</span>}
   {st.type === 'error' &&
    <span className="error">Error!</span>}
   {st.type === 'ready' &&
    <span className="success">{state.text}</span>}
  </span>
 );
};
```

Called **every** prop/state change
(fun fact: it's called twice?! in dev mode)

useEffect for mount 🥴
(aka: oops, I DDOS'd myself by adding a dependency)

4 **Eliminate** all my personal pet peeves with React

Gooey global state

```
const numClicks = field(0);

const ClickCounter = () => {
 return (
  <button
   on:click={() => numClicks.set(
    numClick
   )}
  >
   Clicked {numClicks} times
  </button>
 );
}
```

Easy, free global state!

React global state

Sorry, use Flux

Err wait, use Redux

Hold up, Redux isn't cool anymore, use react-query

Just rearchitect your app every few years

4 **Eliminate** all my personal pet peeves with React

Gooey lists

```
type TodoItem = { done: Field<boolean>; label: string };

const TodoItems = () => {
 const items = collection<TodoItem>([]);
 return (
   <div>
     <ul>
       {items.mapView((item) => (
         <li>
           <input
             type="checkbox"
             checked={item.done}
             on:click={() => item.done.set(
               !item.done.get()
             )}
           />
           {item.label}
         </li>
       ))}
     </ul>
     <ItemForm onNewItem={(item) => items.push(item)} />
   </div>
 );
};
```

Look, no key!

Collections act like arrays!

React lists

```
type TodoItem = { done: boolean; key: string; label: string }

const TodoItems = () =>
 const [items, setItem
 return (
   <div>
     <ul>
       {items.map((item) => (
         <li key={item.key}>
           <input
             type="checkbox"
             checked={item.done}
             onClick={() =>
               setItems(
                 items.map((other) =>
                   item === other
                     ? { ...other, done: !item.done }
                     : other
                 )
               )
             }
           />
           {item.label}
         </li>
       ))}
     </ul>
     <ItemForm
       onAdd={(item) => setItems([...items, item])}
     />
   </div>
 );
};
```

Forgot the key param? Now it's expensive!



4 **Eliminate** all my personal pet peeves with React

Gooey derived state

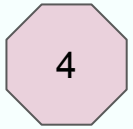
```
const RectCalculator = () => {
 const w = field(0);
 const h = field(0);
 const area = calc(() => w.get() * h.get());
 const perimeter = calc(
   () => 2 * w.get() + 2 * h.get()
 );
 return (
   <div>
     <NumberInput value={w} label="Width" />
     <NumberInput value={h} label="Height" />
     <p>Area: {area} sq ft</p>
     <p>Perimeter: {perimeter} ft</p>
   </div>
 );
};
```

Auto-memoized calc()!

React derived state

```
const RectCalculator = () => {
 const [w, setW] = useState(0);
 const [h, setH] = useState(0);
 const area = useMemo(() => w * h, [w, h]);
 const perimeter = useMemo(() => 2 * w + 2 * h, [w, h]);
 return (
   <div>
     <NumberInput val={w} setVal={setW} label="Width" />
     <NumberInput val={h} setVal={setH} label="Height"
/>
     <p>Area: {area} sq ft</p>
     <p>Perimeter: {perimeter} ft</p>
   </div>
 );
};
```

List all dependencies 😉



Eliminate all my personal pet peeves with React

All I'm trying to say is: making a framework can be fun

Thanks for coming!

(do less work)



Sufian Rhazi

NY UX Speed and Web Performance Meetup

Sep 17, 2026


If you want fast, use SolidJS
SOLIDJS
Simple and performant reactivity for building user interfaces.
Get Started
Intro to Solid (100 seconds)
Advanced intro (5 minutes)